

Model Free
Dynamics of
Env.



Model Based RL

low-cost
① $\underline{P(s'|s, a)}$

② try to fit
 $P(s'|s, a)$ and

then
RL using
this $\hat{P}$.

Deep Reinforcement Learning

Mohammad Hossein Rohban, Ph.D.

Courtesy: Some slides are adopted from CS 285 Berkeley, and CS 234 Stanford, and Pieter Abbeel's compact series on RL.

$\underline{P(s'|s, a)}$

Q, V, π



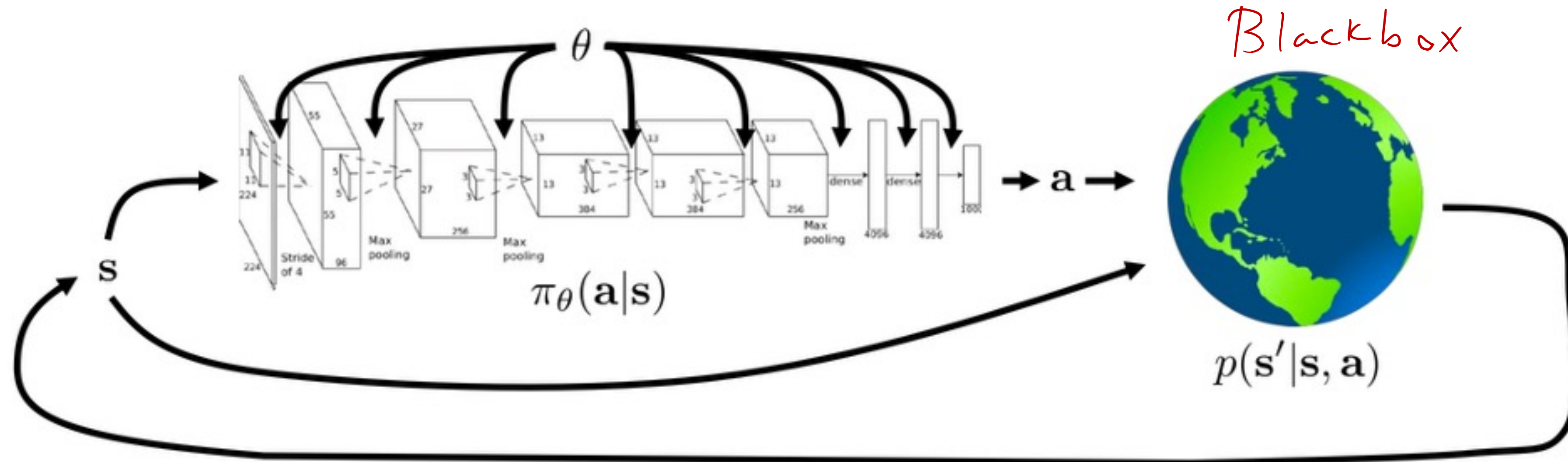
RIML Lab, CE Department, Sharif
University of Technology

Spring 2026

Overview

- ♦ Introduction to model-based reinforcement learning
- ♦ What if we know the dynamics? How can we make decisions?
- ✓ ♦ Stochastic optimization methods
- ✓ ♦ Monte Carlo tree search (MCTS) → AlphaGo
- ♦ Trajectory optimization
- ♦ Goal: Understand how we can perform planning with known dynamics models in discrete and continuous spaces

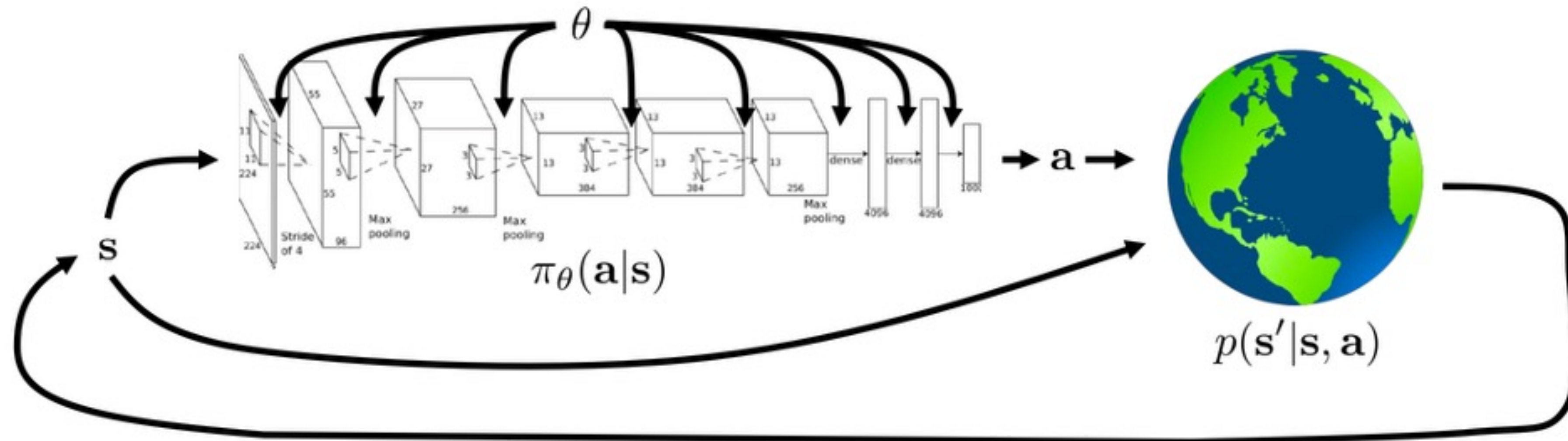
Recap: Model-Free RL



$$\underbrace{p_{\theta}(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_T, \mathbf{a}_T)}_{\pi_{\theta}(\tau)} = p(\mathbf{s}_1) \prod_{t=1}^T \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$

$$\theta^* = \arg \max_{\theta} E_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

Recap: Model-Free RL



$$\underbrace{p_{\theta}(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_T, \mathbf{a}_T)}_{\pi_{\theta}(\tau)} = p(\mathbf{s}_1) \prod_{t=1}^T \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) \cancel{p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)}$$

$$\theta^* = \arg \max_{\theta} E_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

What if we knew the transition dynamics?

♦ Often we do know the dynamics

- ☑ Games (e.g., Atari games, chess, Go)
- ☑ Easily modeled systems (e.g., navigating a car)
- ☑ Simulated environments (e.g., simulated robots, video games)

♦ Often we can learn the dynamics

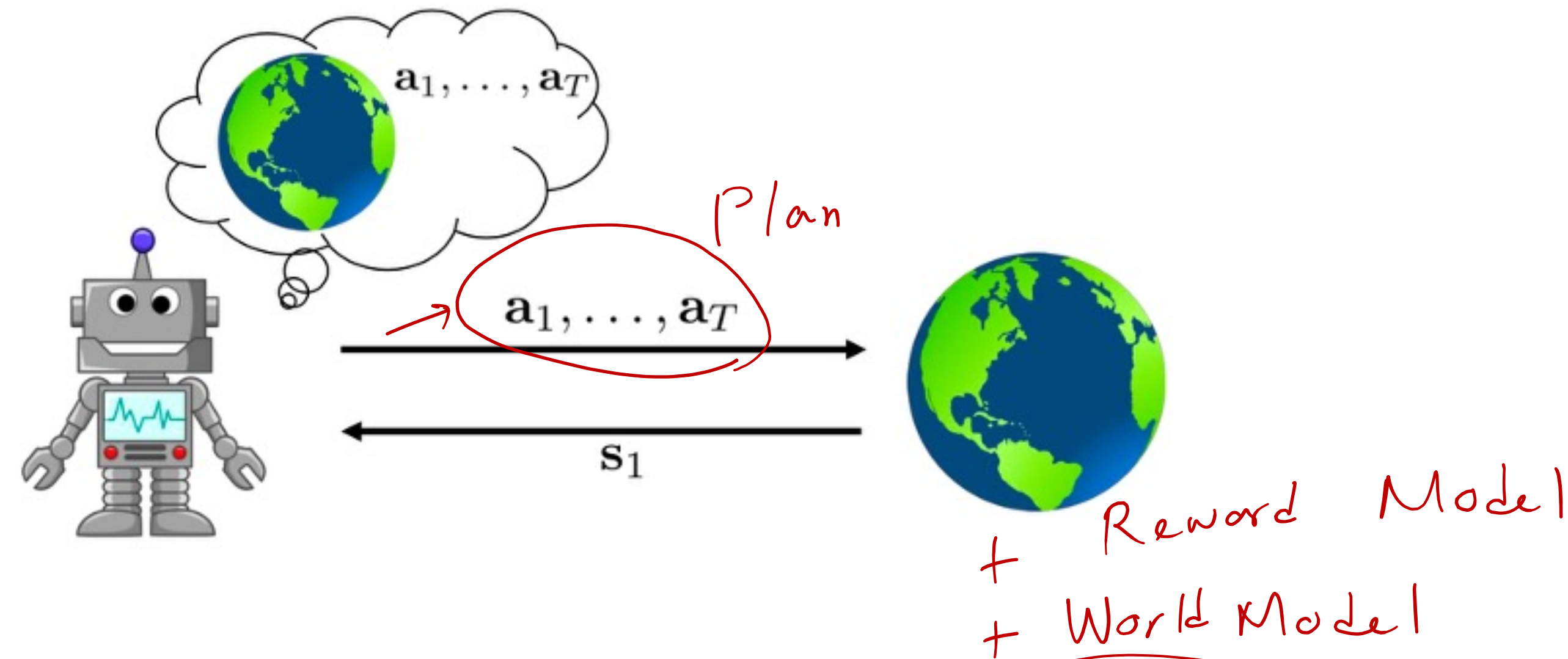
- ☑ System identification – fit unknown parameters of a known model
- ☑ Learning – fit a general-purpose model to observed transition data

Does knowing the dynamics make things easier? Often, yes!

Model-Based RL

- ✦ Model-based reinforcement learning: learn the **transition dynamics**, then figure out how to choose actions.
- ✦ Today: how can we make decisions if we know the dynamics?
 - ☑ a. How can we choose actions under **perfect knowledge** of the system dynamics?
 - ☑ b. Optimal control, trajectory optimization, planning

The deterministic case

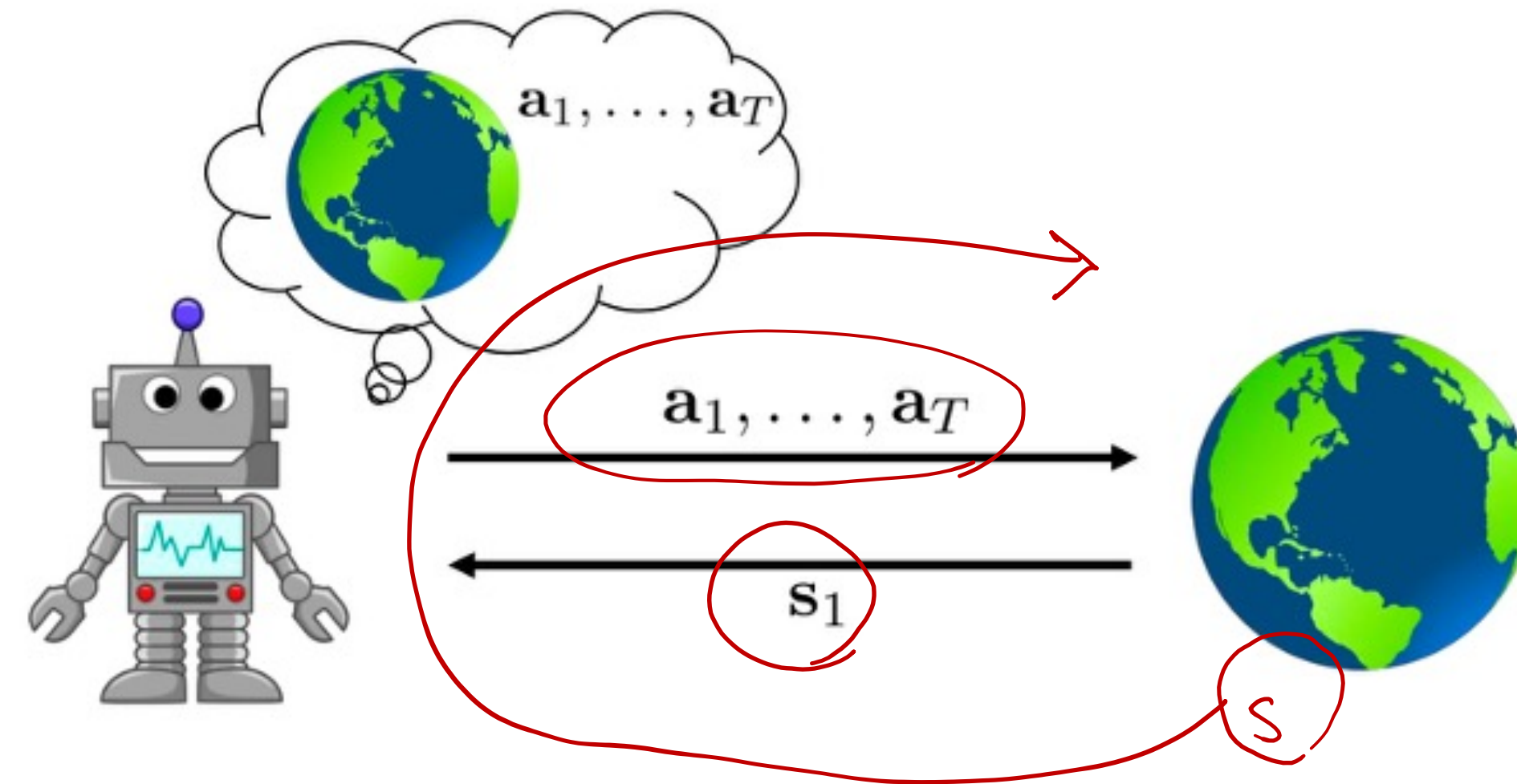


$$\mathbf{a}_1, \dots, \mathbf{a}_T = \arg \max_{\mathbf{a}_1, \dots, \mathbf{a}_T} \sum_{t=1}^T r(\mathbf{s}_t, \mathbf{a}_t) \text{ s.t. } \underline{\mathbf{s}_{t+1}} = f(\mathbf{s}_t, \mathbf{a}_t)$$

Constrained
Opt.

Why actions & not policy?

The stochastic open-loop case



$$p_{\theta}(s_1, \dots, s_T | a_1, \dots, a_T) = p(s_1) \prod_{t=1}^T p(s_{t+1} | s_t, a_t)$$

Suboptimal ←

$$\underline{a_1, \dots, a_T} = \arg \max_{a_1, \dots, a_T} E \left[\sum_t r(s_t, a_t) | a_1, \dots, a_T \right]$$

The stochastic open-loop case (cont.)

Scenario 2

+

$a_1 \dots a_T$

کری می خواست به عیادت بیماری برود. اندیشید که هنگام احوال پرسی ممکن است صدای اورانشنوم و پاسخ ناخایسته بدهم. ازاین رودری چاره برآمد و بالاخره باخود گفت: بهتر است پرسشها را پیش از رفتن بسنجم و پاسخ رانیز برآورد کنم تا دچار اشتباه نشوم.

بنابراین پرسشهای خود را چنین پیش بینی کرد:

- ابتدا از اومی پرسم حالت بهتر است؟ او خواهد گفت "آری" من در جواب می گویم: خدا را شکر

- بعد از اومی پرسم چه خورده ای؟ لابد نام غذایی را خواهد آورد. من می گویم گوارا باد.

- در پایان می پرسم پزشکی کیست؟ نام پزشکی را می گوید و من پاسخ می دهم: مقدمش مبارک باد.

.....

چون به خانه ی بیمار رسید همان گونه که از پیش آماده شده بود به احوال پرسی پرداخت:

- کر گفت: "چگونه ای؟"

بیمار گفت: مُردم

کر گفت: خدا را شکر

بیمار از این سخن بیجا برآشفست.

- بعد از آن پرسید: "چه خورده ای؟"

بیمار گفت: زهر

کر گفت: گوارا باد. داروی خوبی است.

بیمار از این پاسخ نیز بیشتر به خود پیچید.

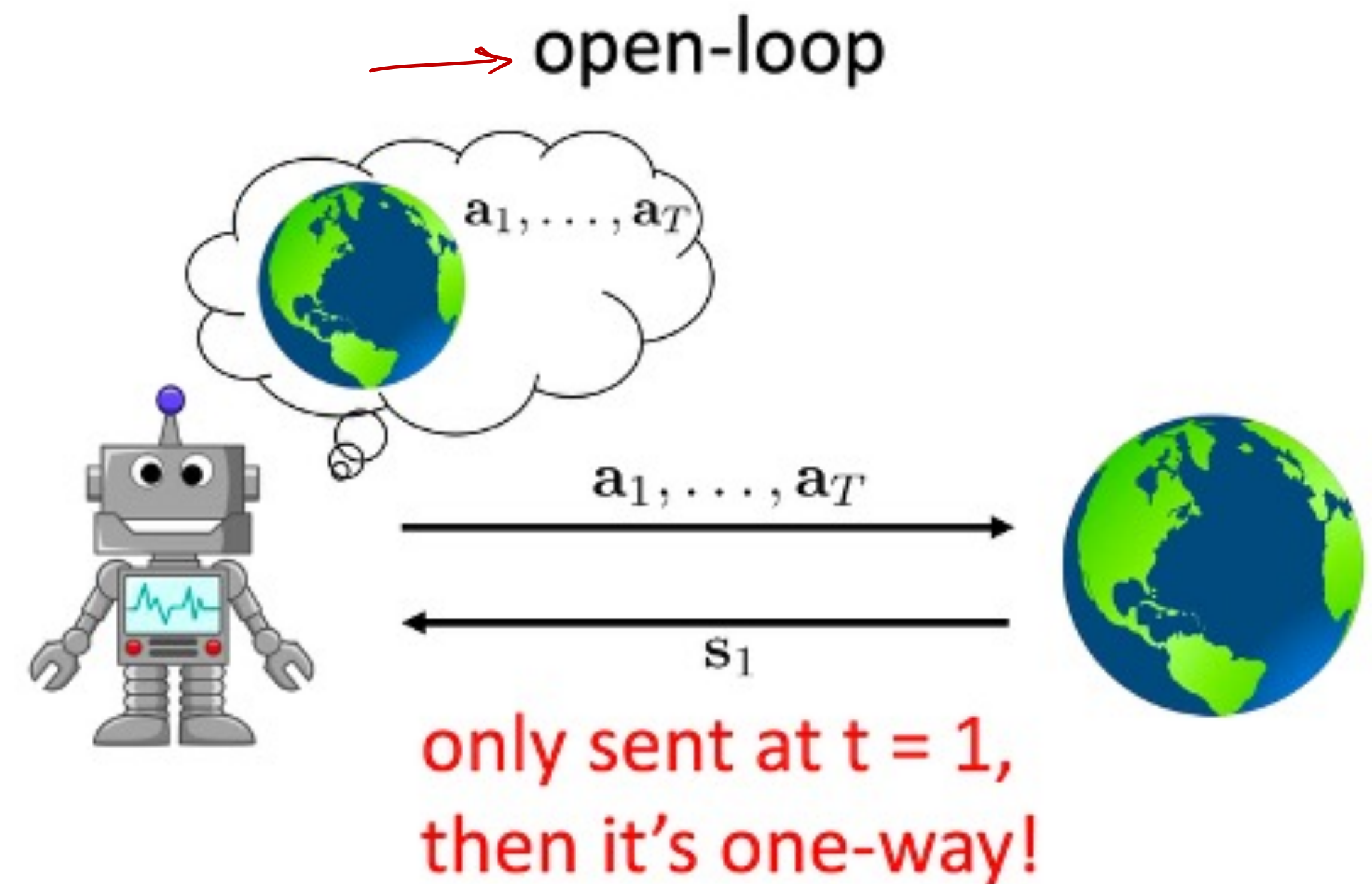
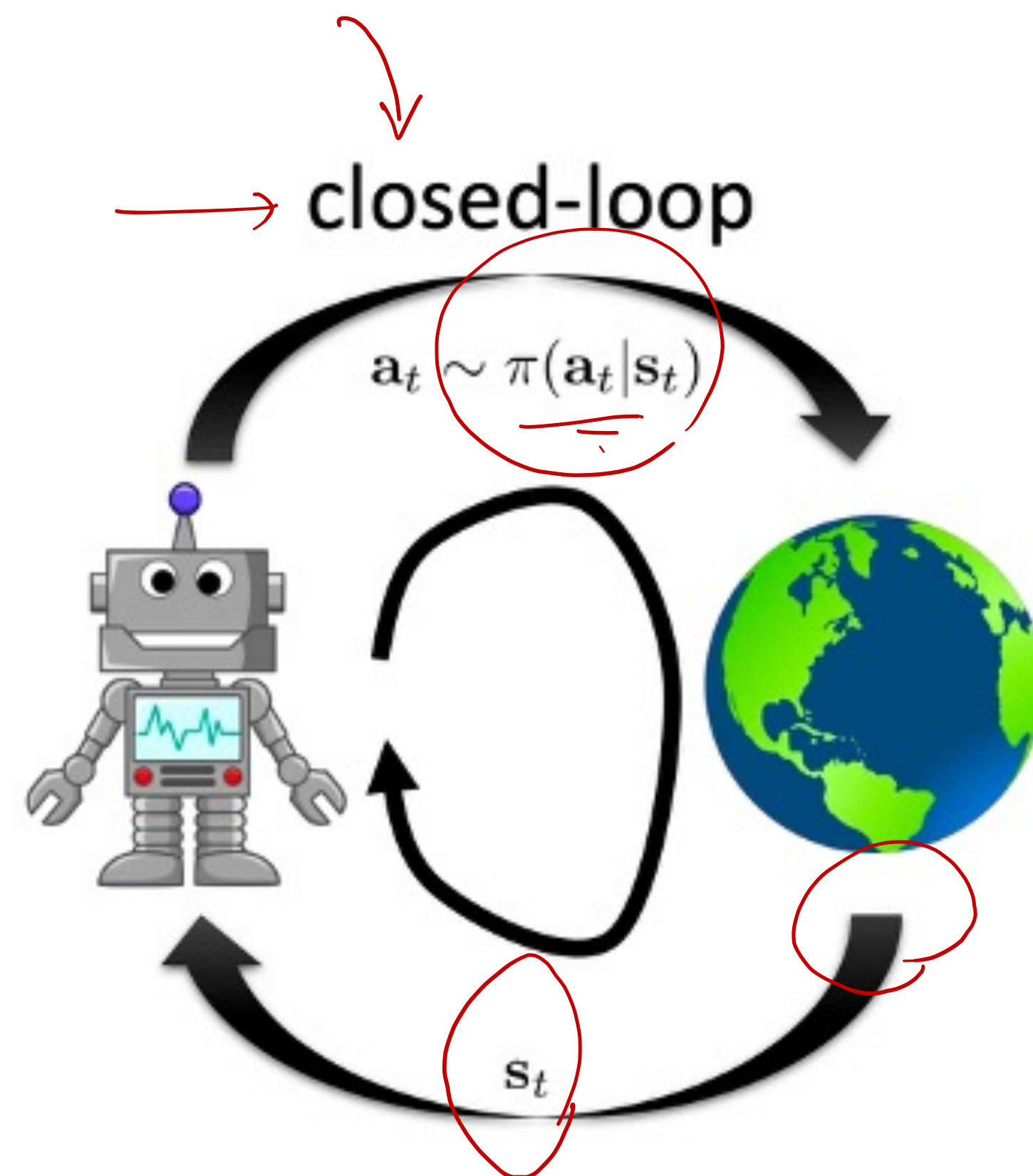
- بعد از آن کر گفت: "از طبیبان کیست او" کاوهمی آید به چاره پیش تو؟"

بیمار که آشفتگی و ناراحتی اش به نهایت رسیده بود در پاسخ گفت:

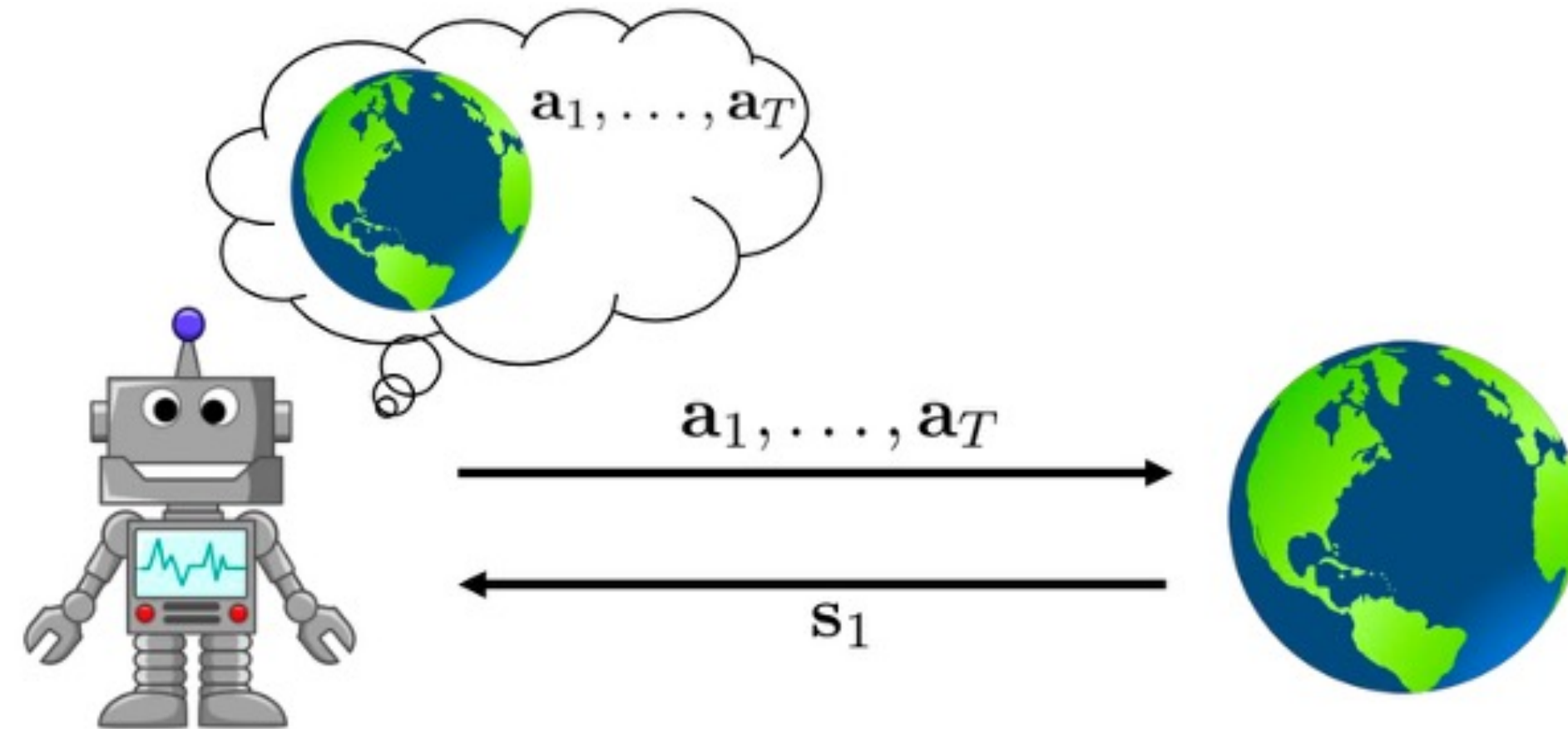
عزرائیل می آید، برو.

کر گفت: پایش بس مبارک. شاد شو!

The stochastic open-loop case (cont.)



The stochastic open-loop case (cont.)



Form of π ?
Neural Net

GLOBAL

Time varying linear
 $K_t s_t + k_t$

LOCAL

$$p(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_T, \mathbf{a}_T) = p(\mathbf{s}_1) \prod_{t=1}^T \pi(\mathbf{a}_t | \mathbf{s}_t) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$

$$\pi = \arg \max_{\pi} E_{\tau \sim p(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

Stochastic Optimization

Unconstrained Opt.

$$J = \sum_{i=1}^T r(s_i, a_i)$$

♦ abstract away optimal control/planning:

$$\mathbf{a}_1, \dots, \mathbf{a}_T = \arg \max_{\mathbf{a}_1, \dots, \mathbf{a}_T} \underbrace{J(\mathbf{a}_1, \dots, \mathbf{a}_T)}_{\text{don't care what this is}} \quad \mathbf{A} = \arg \max_{\mathbf{A}} J(\mathbf{A})$$

possible plan

$$\mathbf{A}_1 = (a_{1,1} \dots a_{1,T}) \dots$$

$$\mathbf{A}_N = (a_{N,1} \dots a_{N,T})$$

♦ simplest method: guess & check

“random shooting method”

$$\rightarrow \mathcal{N}(\mu, \Sigma)$$

1. pick $\mathbf{A}_1, \dots, \mathbf{A}_N$ from some distribution (e.g., uniform) $\xrightarrow{\text{MLE}}$

2. choose $\mathbf{A}_i$ based on $\arg \max_i J(\mathbf{A}_i)$

$$\hat{\mu} = \frac{1}{N} \sum_{i=1}^N \mathbf{A}_i$$

$$\hat{\Sigma} = \frac{1}{N-1} \sum_{i=1}^N (\mathbf{A}_i - \hat{\mu})(\mathbf{A}_i - \hat{\mu})^T$$

CEM (Cross Entropy Method)

$M=2$

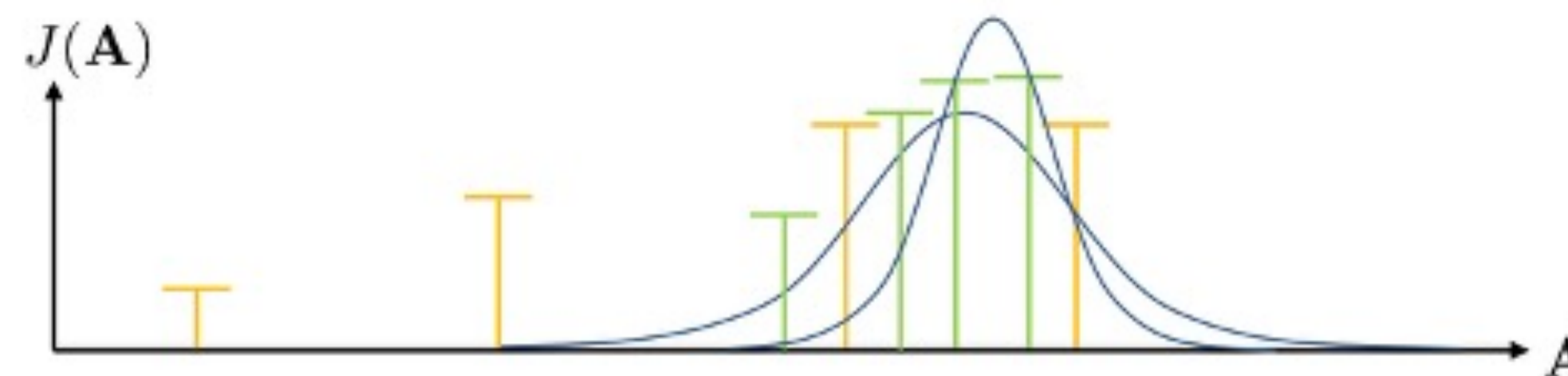
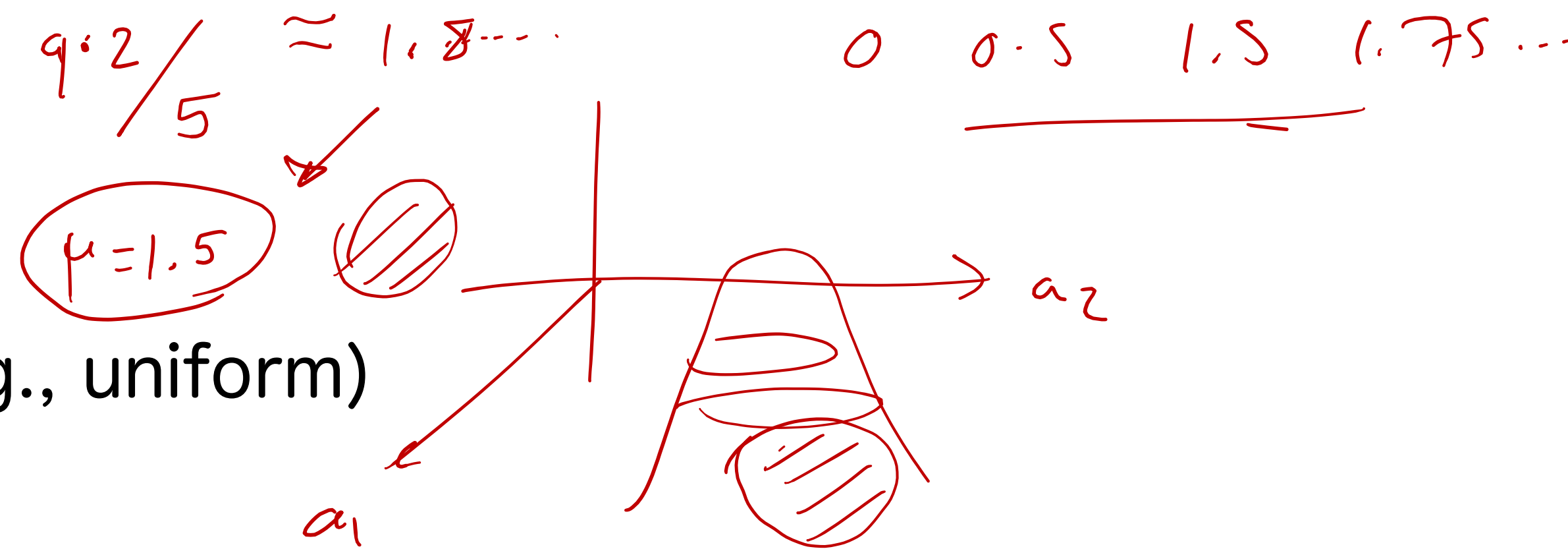
J $\overset{a}{\overset{1}{10}}$ $\overset{2}{5}$ $\overset{3.2}{-2}$ $\overset{4}{-5}$ $\overset{-1}{0}$

1. pick $\mathbf{A}_1, \dots, \mathbf{A}_N$ from some distribution (e.g., uniform)
2. choose $\mathbf{A}_i$ based on $\text{argmax}_i J(\mathbf{A}_i)$

Planning

← for

Can we do better?



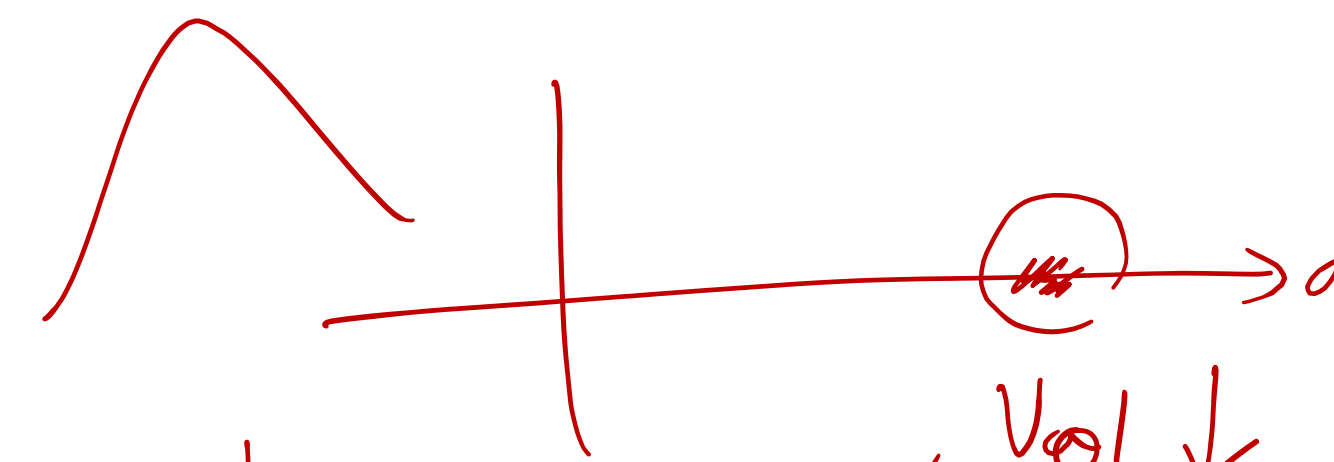
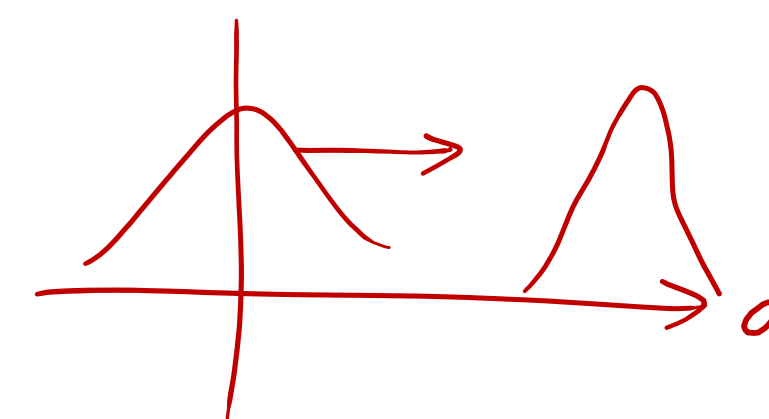
cross-entropy method with continuous-valued inputs:

1. sample $\mathbf{A}_1, \dots, \mathbf{A}_N$ from $p(\mathbf{A})$
2. evaluate $J(\mathbf{A}_1), \dots, J(\mathbf{A}_N)$
3. pick the elites $\mathbf{A}_{i_1}, \dots, \mathbf{A}_{i_M}$ with the highest value, where $M < N$
4. refit $p(\mathbf{A})$ to the elites $\mathbf{A}_{i_1}, \dots, \mathbf{A}_{i_M}$

→ Parallel

Blind local

GMM + EM



horizon $\equiv T \equiv \text{dim} \uparrow$

Pros and Cons!

♦ Pros

- ☑ Could be very fast (Parallelizable)
- ☑ Extremely simple

♦ Cons

- ☑ Very harsh dimensionality limit
- ☑ Only open-loop planning

$$\underline{Q(s, a)}$$

Value Iteration

≡ planning

$$\begin{array}{c} P(s'|s, a) \\ R(s, a) \\ \vdots \\ \downarrow \\ \pi \end{array}$$

How to extend to the
Closed-loop setting?

Model
pred.
Control
(MPC)

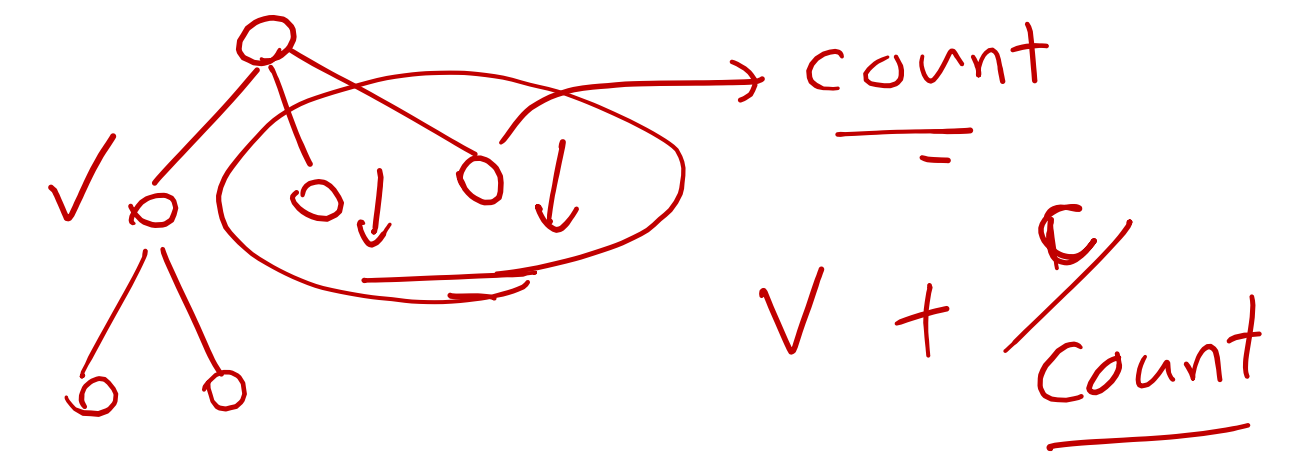
replanning
→ $a_1 \dots a_T$

execute

$$a_1^* \dots a_T^* = \arg \max_{a_1 \dots a_T} J(a_1 \dots a_T)$$

a_1^* → observe the actual ^{next} state $\underbrace{S_2}$

Discrete Case: Monte Carlo Tree Search

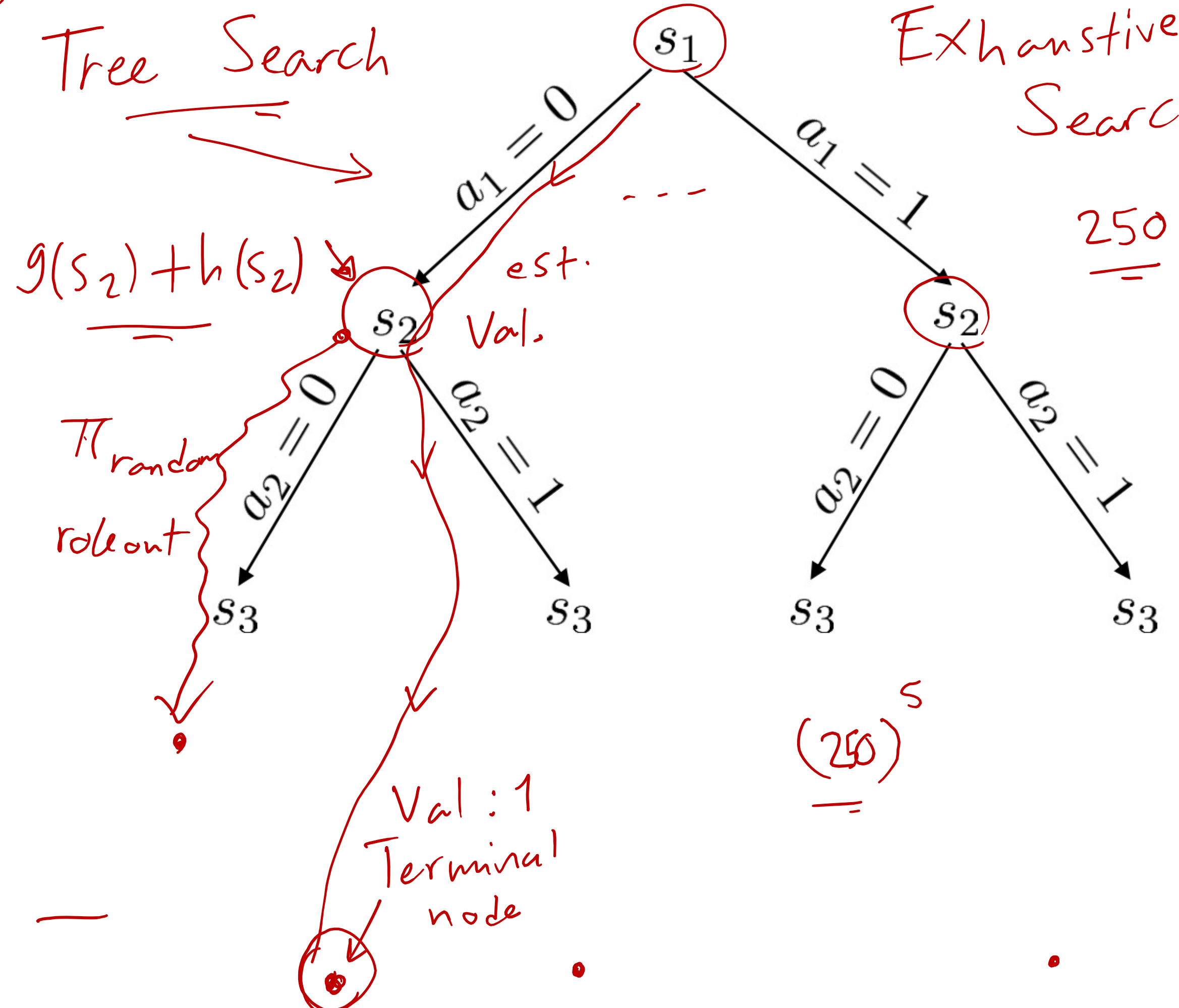


discrete planning as a search problem



Tree Search

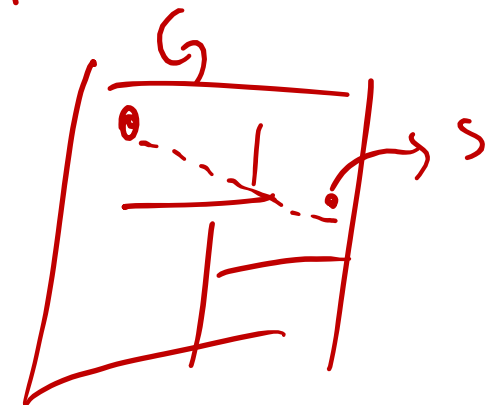
Exhaustive Search



Val. func.

heuristic function

precise s_t → bad. Cost
 prioritization



T

Discrete Case: Monte Carlo Tree Search (cont.)

how to approximate value without full tree?

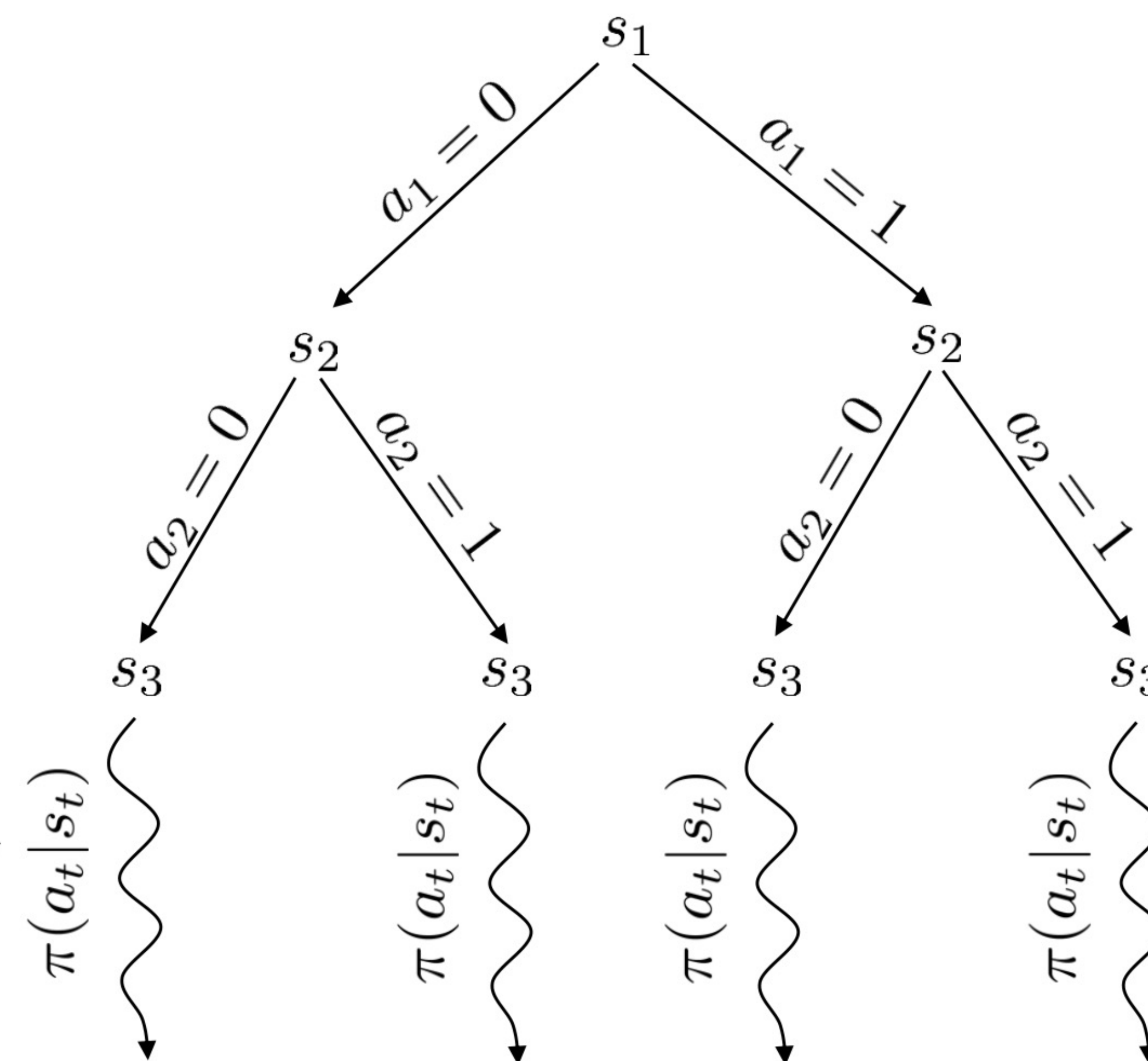


s_t



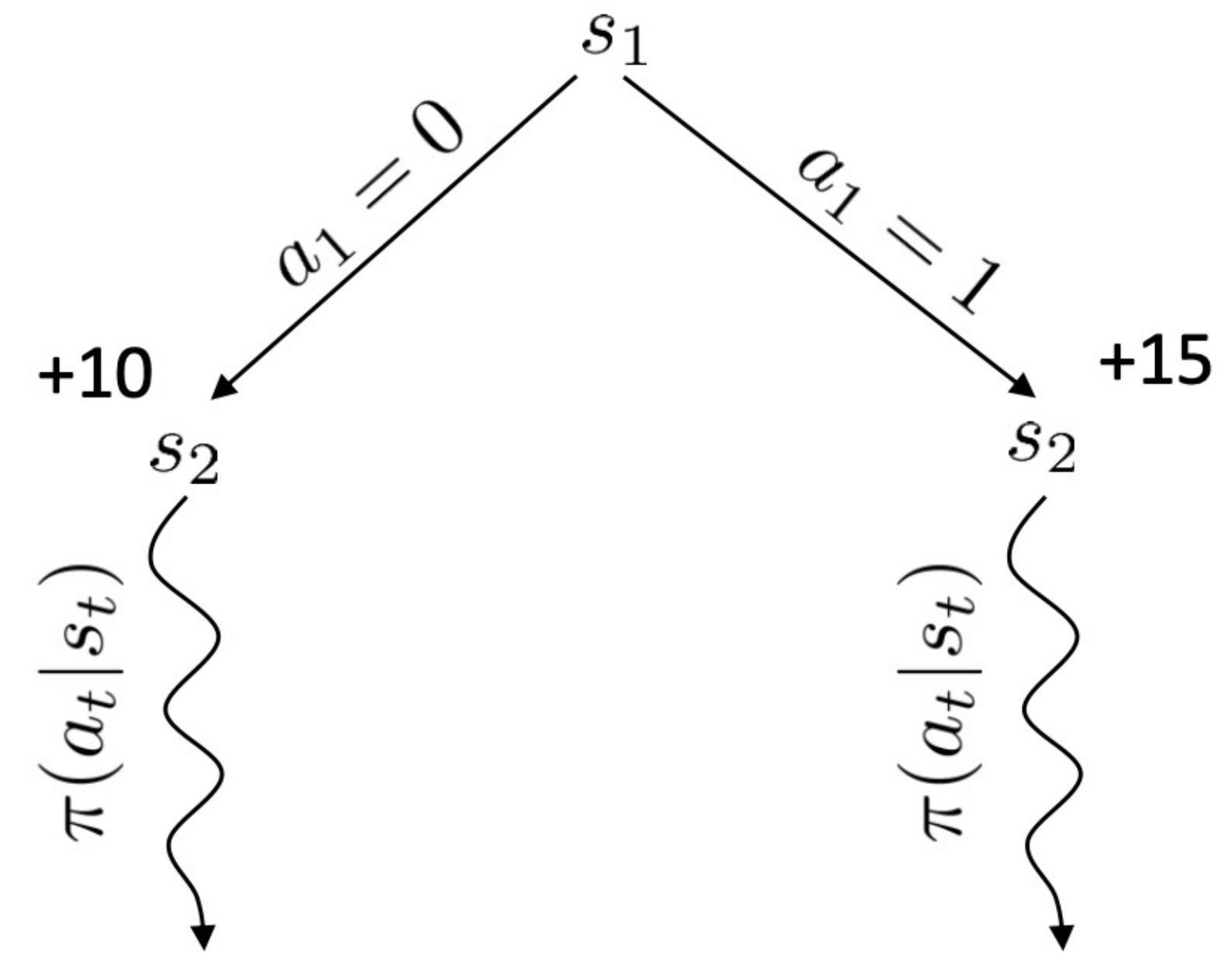
a_t

e.g., random policy



Discrete Case: Monte Carlo Tree Search (cont.)

can't search all paths – where to search first?



intuition: choose nodes with best reward, but also prefer rarely visited nodes

Discrete Case: Monte Carlo Tree Search (cont.)

Tree Search Algorithm

generic MCTS sketch

1. find a leaf s_l using $\text{TreePolicy}(s_1)$
2. evaluate the leaf using $\text{DefaultPolicy}(s_l)$
3. update all values in tree between s_1 and s_l

take best action from s_1



TreePolicy Function

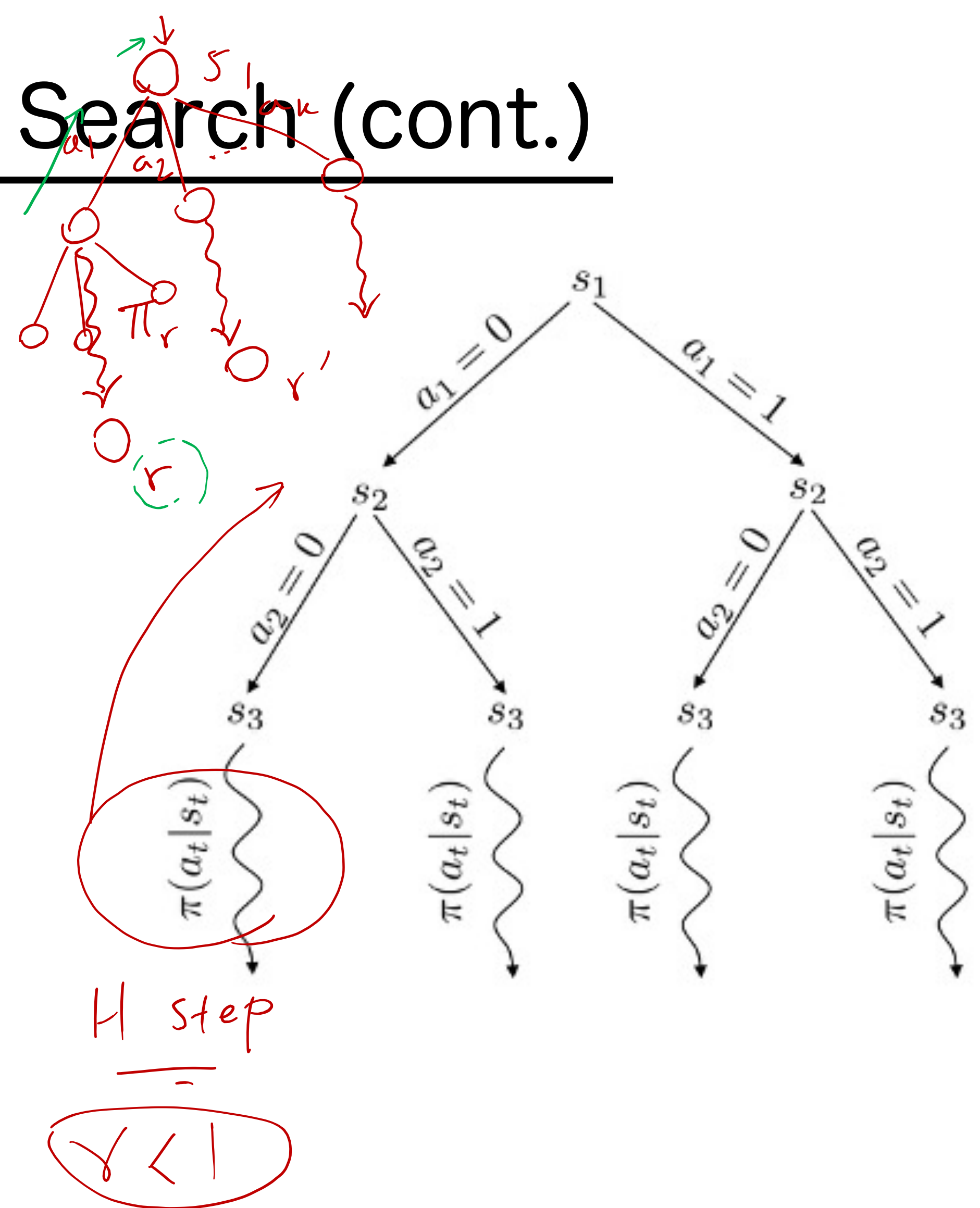
UCT $\text{TreePolicy}(s_t)$

if s_t not fully expanded, choose new a_t

else choose child with best $\text{Score}(s_{t+1})$

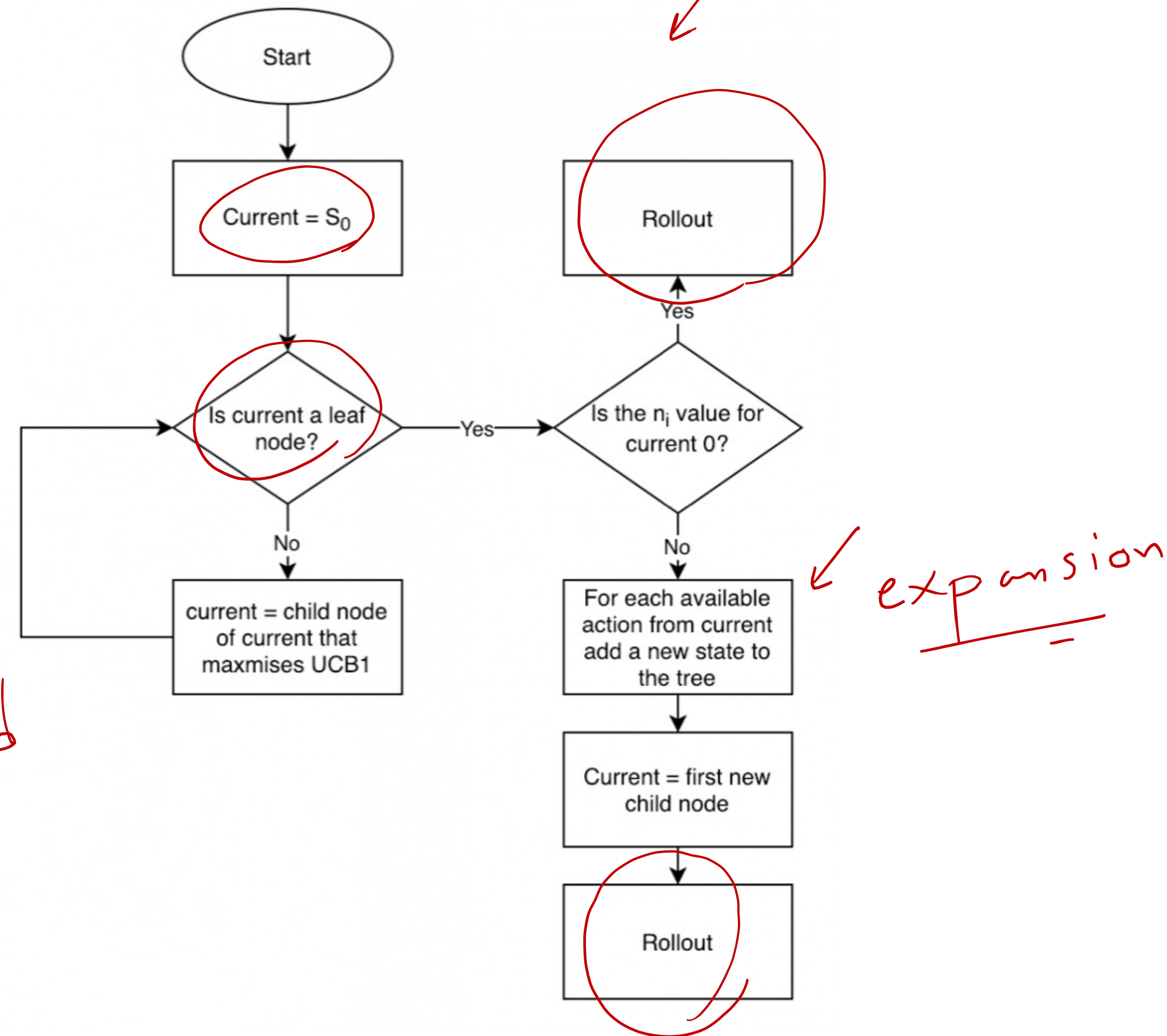
$$\text{Score}(s_t) = \frac{Q(s_t)}{N(s_t)} + 2C \sqrt{\frac{2 \ln N(s_{t-1})}{N(s_t)}}$$

UCB



Discrete Case: Monte Carlo Tree Search (cont.)

leaf: a node
whose children
are not expanded
yet. its n could
be non-zero



Additional Reading

- ♦ Browne, Powley, Whitehouse, Lucas, Cowling, Rohlfshagen, Tavener, Perez, Samothrakis, Colton. (2012). A Survey of Monte Carlo Tree Search Methods.
 - Survey of MCTS methods and basic summary.

Today's Lecture

$$s_{t+1} \leftarrow \underbrace{f(s_t, a_t)}_{\text{unknown}}$$

1. Basics of model-based RL: learn a model, use model for control

- ✓ Why does naïve approach not work?
- ✓ The effect of distributional shift in model-based RL

Slow
test time

Planning
Opt. (CEM)
online

2. Uncertainty in model-based RL

3. Model-based Policy Learning

↓
π Policy

Goals:

- ✓ Understand how to build model-based RL algorithms
- ✓ Understand the important considerations for model-based RL
- ✓ Understand the tradeoffs between different model class choices

Why learn the Model?

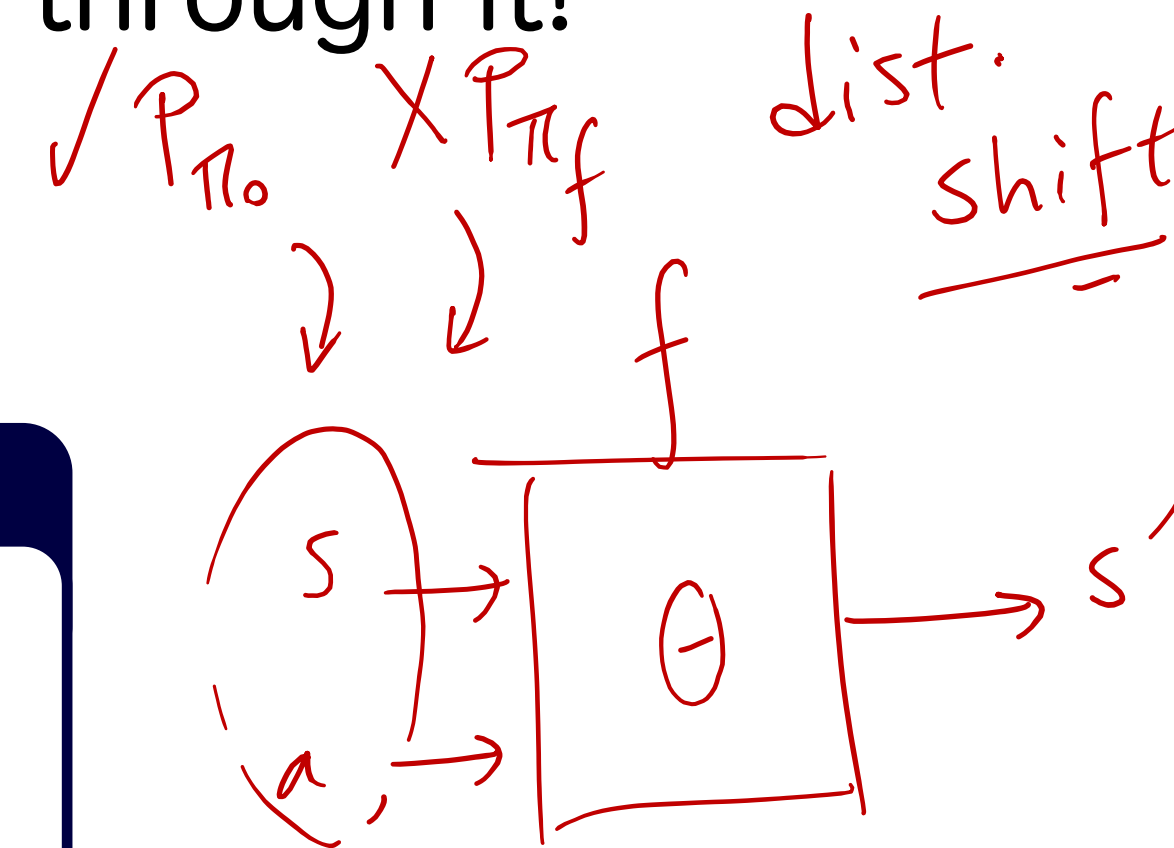
If we knew $f(\mathbf{s}_t, \mathbf{a}_t) = \mathbf{s}_{t+1}$, we could use the tools from last week.

(or $p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$ in the stochastic case)

So let's learn $f(\mathbf{s}_t, \mathbf{a}_t)$ from data, and then plan through it!

Model-Based reinforcement learning v0.5

1. run base policy $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$ (e.g., random policy) to collect $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$
2. learn dynamics model $f(\mathbf{s}, \mathbf{a})$ to minimize $\sum_i \|f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$
3. plan through $\hat{f}(\mathbf{s}, \mathbf{a})$ to choose actions (CEM)



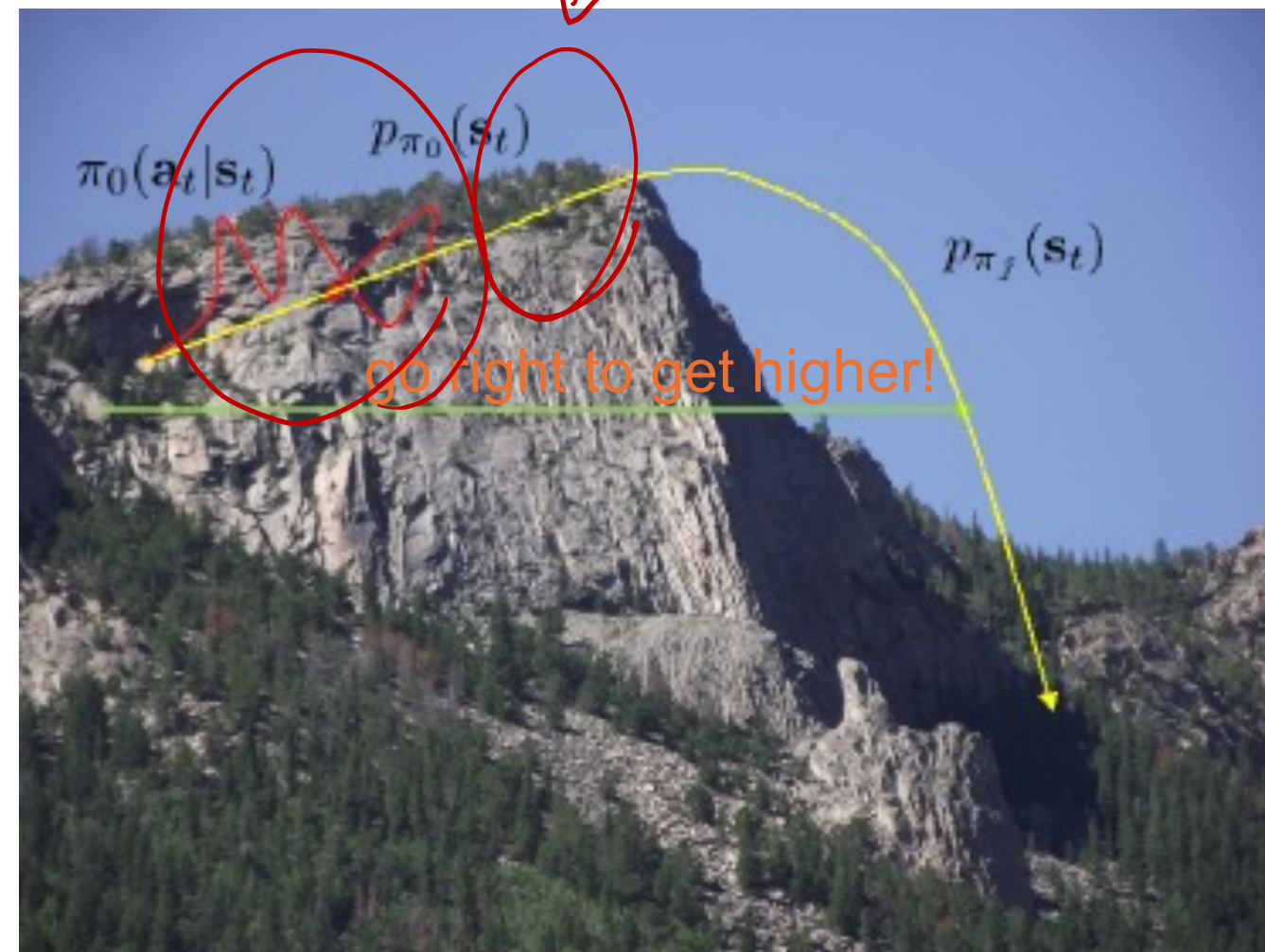
loses @
higher time
horizons

World Model
performs
worse as
planning continues

Does it Work? YES

- ♦ Essentially how system identification works in classical robotics
- ♦ Some care should be taken to design a good base policy
- ♦ Particularly effective if we can hand-engineer a dynamics representation using our knowledge of physics, and fit just a few parameters

Does it Work? (cont.)



$$P_{\pi_f}(s_t) \neq P_{\pi_0}(s_t)$$

Model-Based reinforcement learning v0.5

1. run base policy $\pi_0(a_t|s_t)$ (e.g., random policy) to collect

$$\mathcal{D} = \{(s, a, s')_i\}$$

2. learn dynamics model $f(s, a)$ to minimize $\sum_i \|f(s_i, a_i) - s'_i\|^2$

3. plan through $f(s, a)$ to choose actions $a_1, \dots, a_T$

medium
ranged
horizon

$$s_1, \dots, s_T$$

$$s_1, a_1, s_2, a_2, \dots$$

① ②

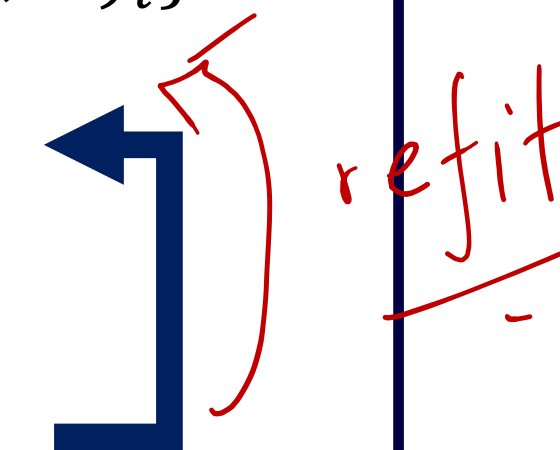
✓ Distribution mismatch problem becomes exacerbated as we use more expressive model classes

Can we do better?

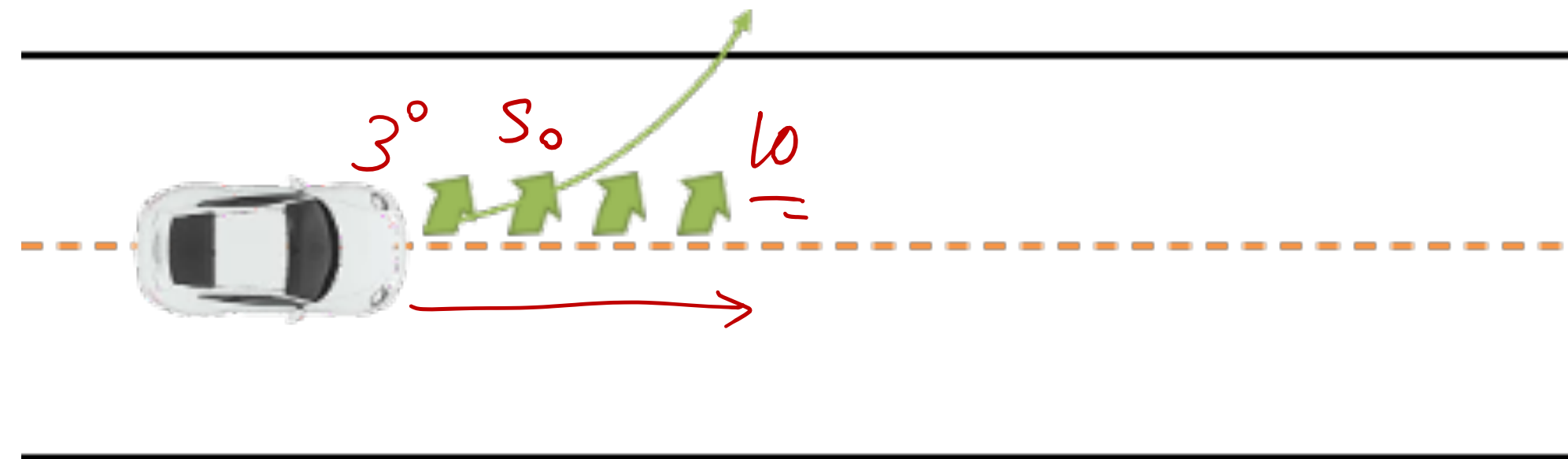
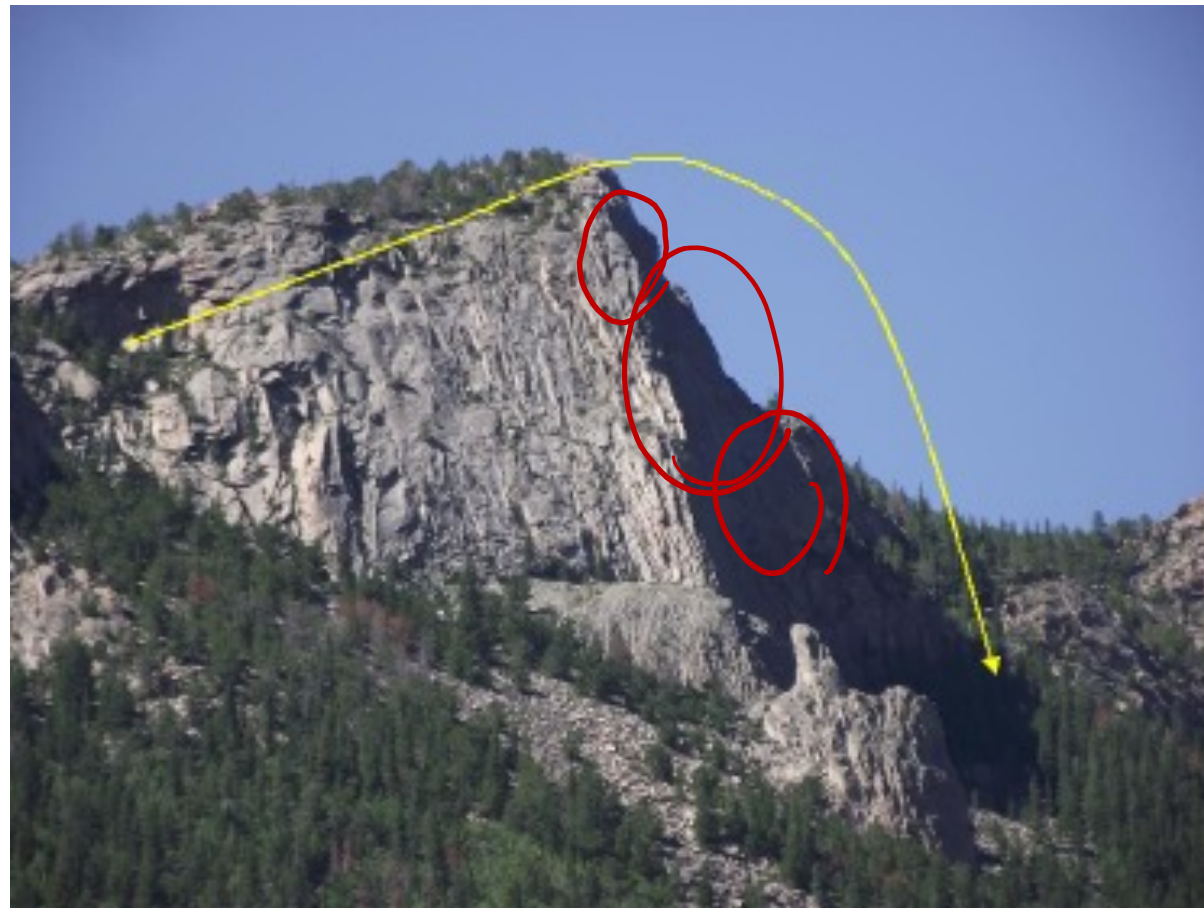
- ♦ can we make $p_{\pi_0}(\mathbf{s}_t) = p_{\pi_f}(\mathbf{s}_t)$?
- ♦ where have we seen that before? need to collect data from $p_{\pi_f}(\mathbf{s}_t)$

Model-Based reinforcement learning v1

1. run base policy $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$ (e.g., random policy) to collect $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$
2. learn dynamics model $f(\mathbf{s}, \mathbf{a})$ to minimize $\sum_i \|f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$
3. plan through $f(\mathbf{s}, \mathbf{a})$ to choose actions
4. execute those actions and add the resulting data $\{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_j\}$ to $\mathcal{D}$



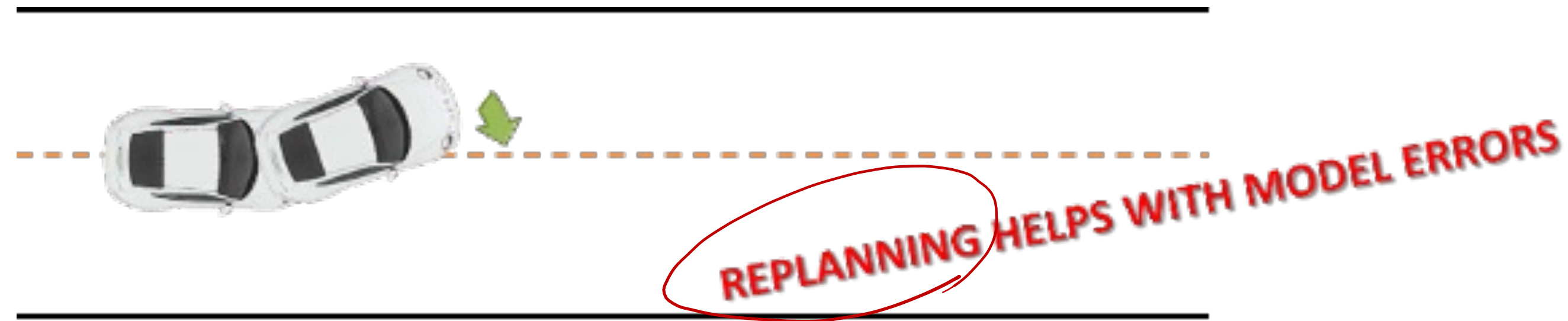
What if we make a MISTAKE?



Sol. : ?
MPC

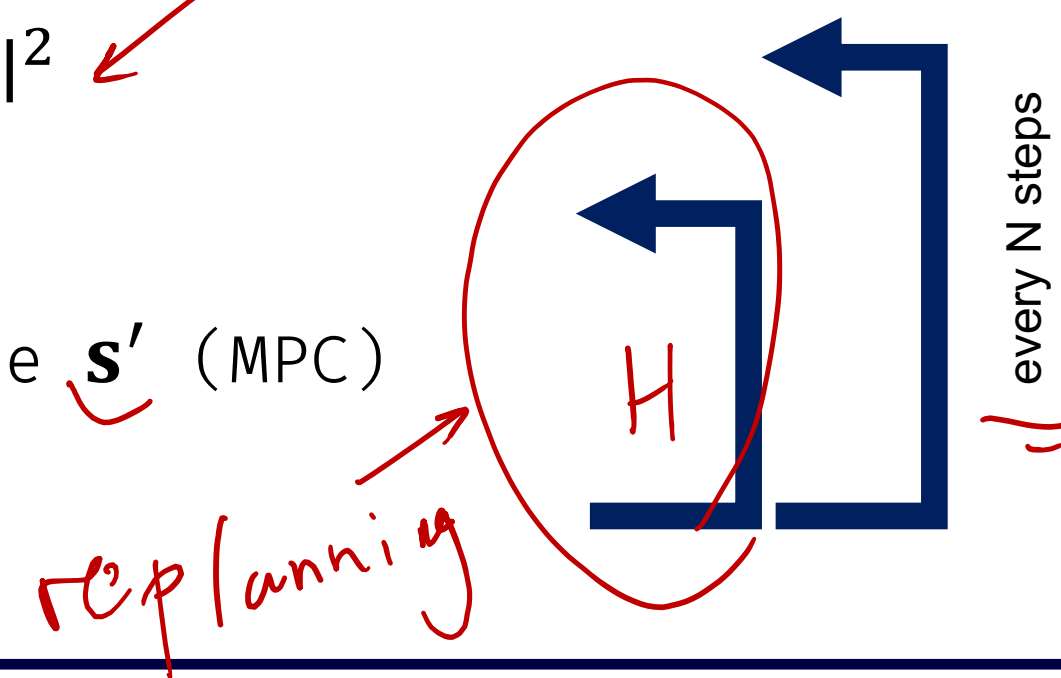
Plan with large H , $a_1, \dots, a_H$, but only execute the 1st action, and observe the resulting state. (closed loop)

Can we do better?



Model-Based reinforcement learning v1.5

1. run base policy $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$ (e.g., random policy) to collect $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$
2. learn dynamics model $f(\mathbf{s}, \mathbf{a})$ to minimize $\sum_i \|f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$ ✓
3. plan through $f(\mathbf{s}, \mathbf{a})$ to choose actions
4. execute the first planned action, observe resulting state $\mathbf{s}'$ (MPC)
5. append $(\mathbf{s}, \mathbf{a}, \mathbf{s}')$ to dataset $\mathcal{D}$



How to Replan?

$\hat{f}$ $\swarrow$ Simple $\rightarrow$ Could not capture the complex dyn.
 $\searrow$ Complex $\rightarrow$ Overfit

Model-Based reinforcement learning v1.5

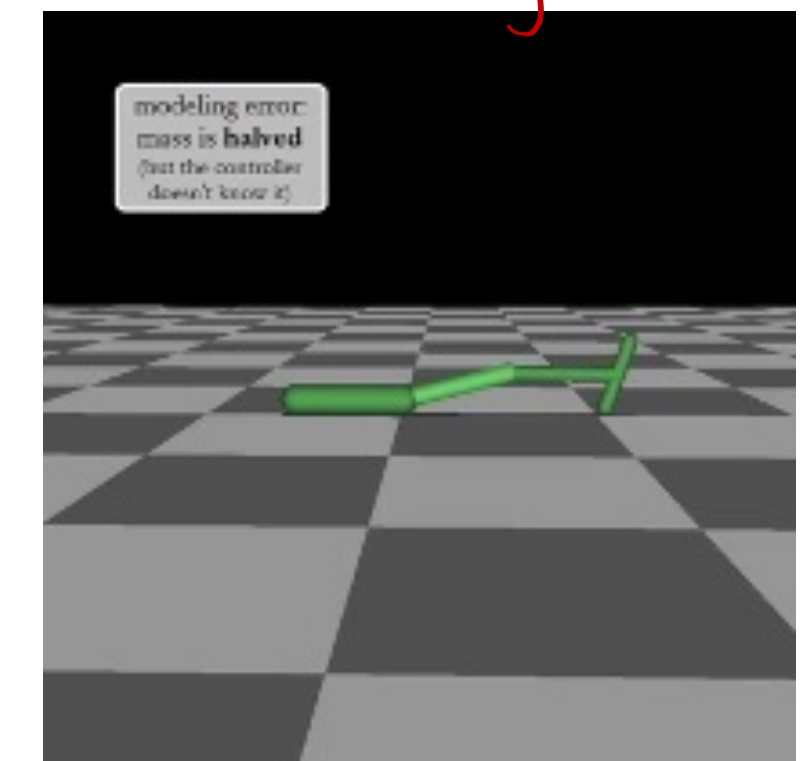
1. run base policy $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$ (e.g., random policy) to collect $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$
2. learn dynamics model $f(\mathbf{s}, \mathbf{a})$ to minimize $\sum_i \|f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$
3. plan through $f(\mathbf{s}, \mathbf{a})$ to choose actions
4. execute the first planned action, observe resulting state $\mathbf{s}'$ (MPC)
5. append $(\mathbf{s}, \mathbf{a}, \mathbf{s}')$ to dataset $\mathcal{D}$



↓ planning based on $\hat{f}$
Junk data
↓
Not be able to improve $\hat{f}$

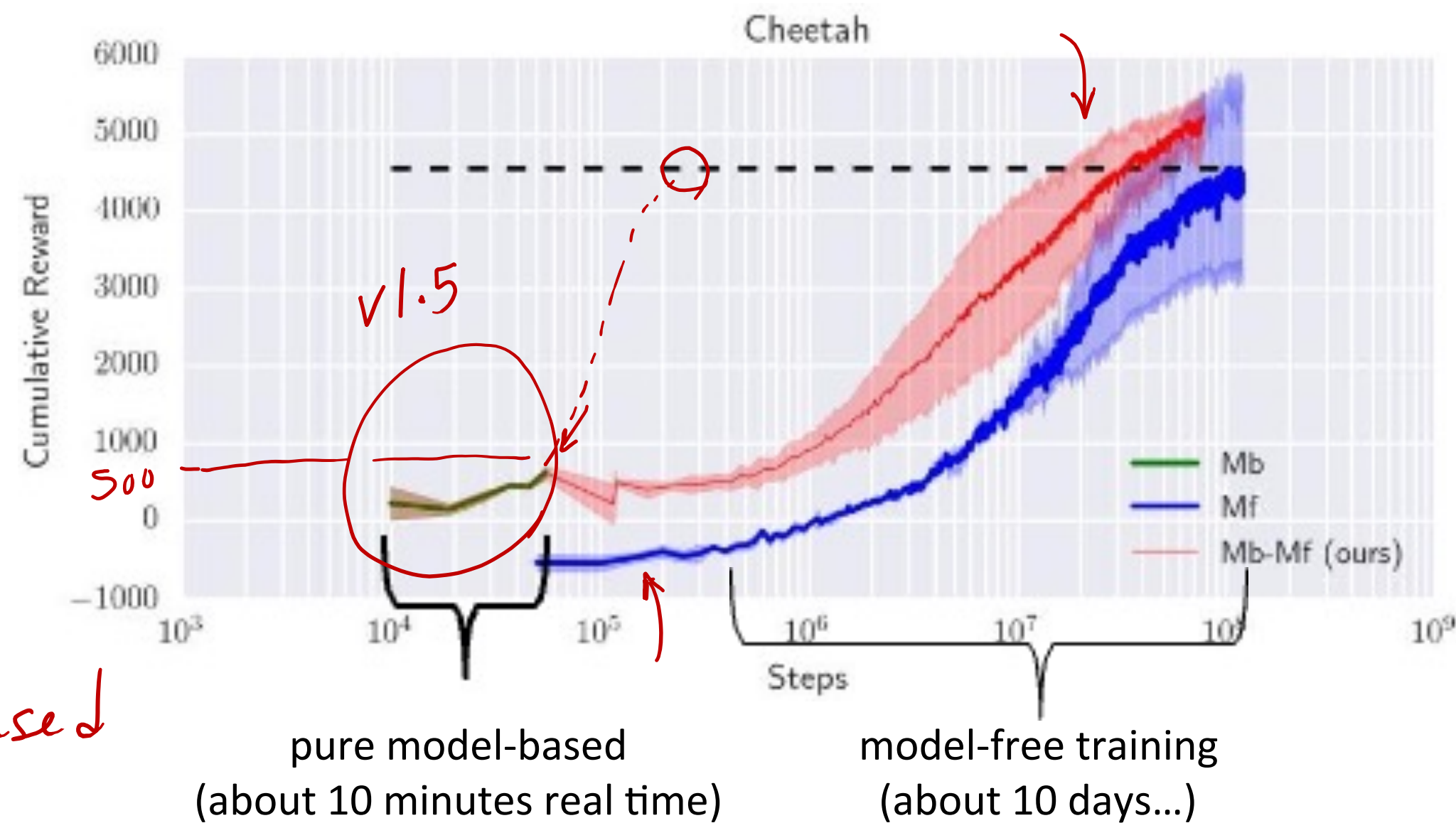
$\sim \hat{f}$ MLP (2-layer)

- ♦ The more you replan, the less perfect each individual plan needs to be
- ♦ Can use shorter horizons
- ♦ Even random sampling can often work well here!

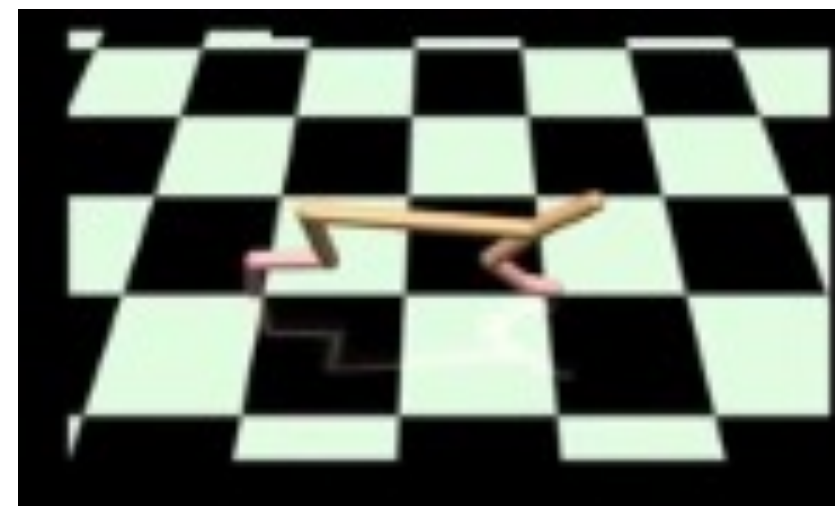


Uncertainty in Model-Based RL

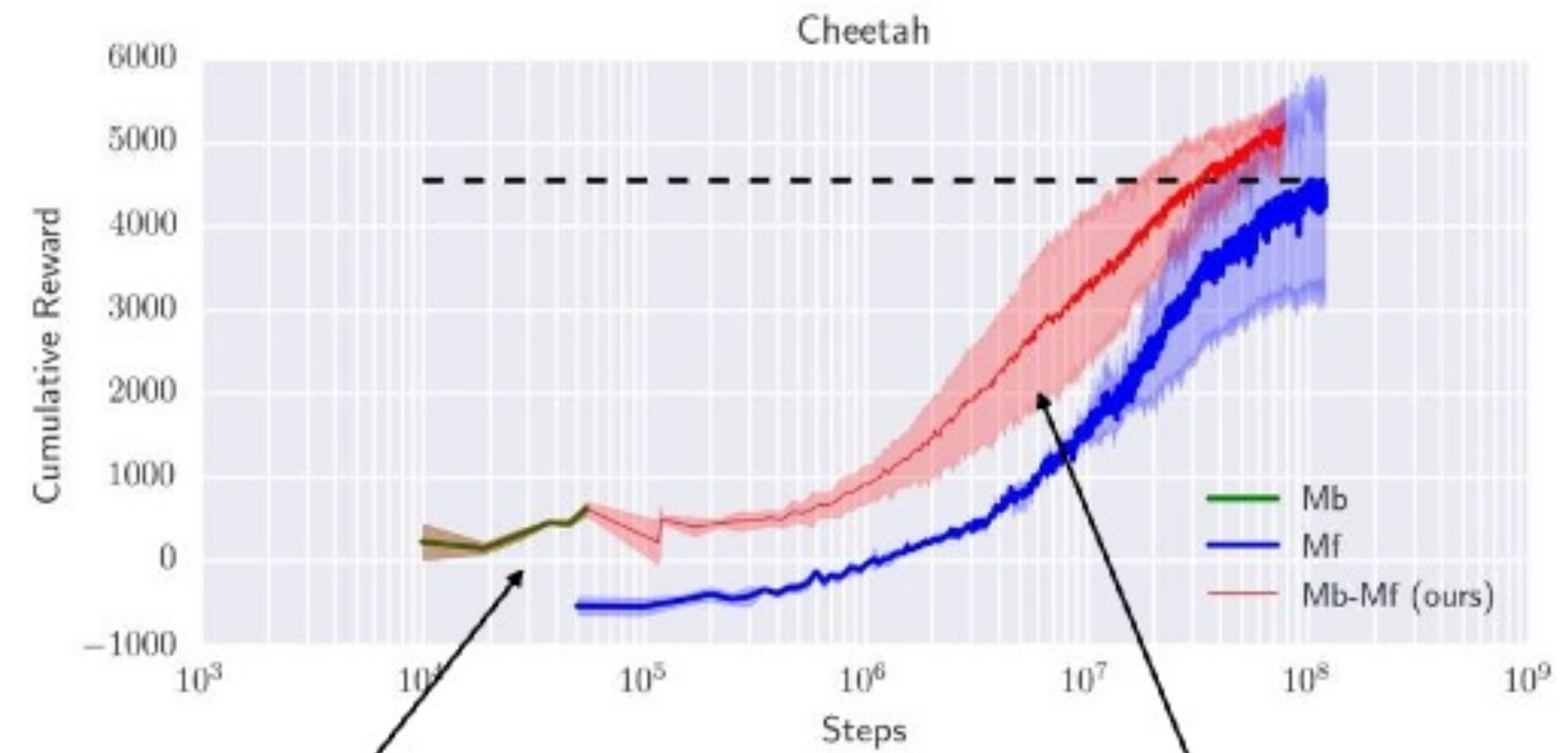
A Performance gap in Model-Based RL



Q: why the pure model-based method failed to reach a high cumulative reward?



Why the Performance gap?



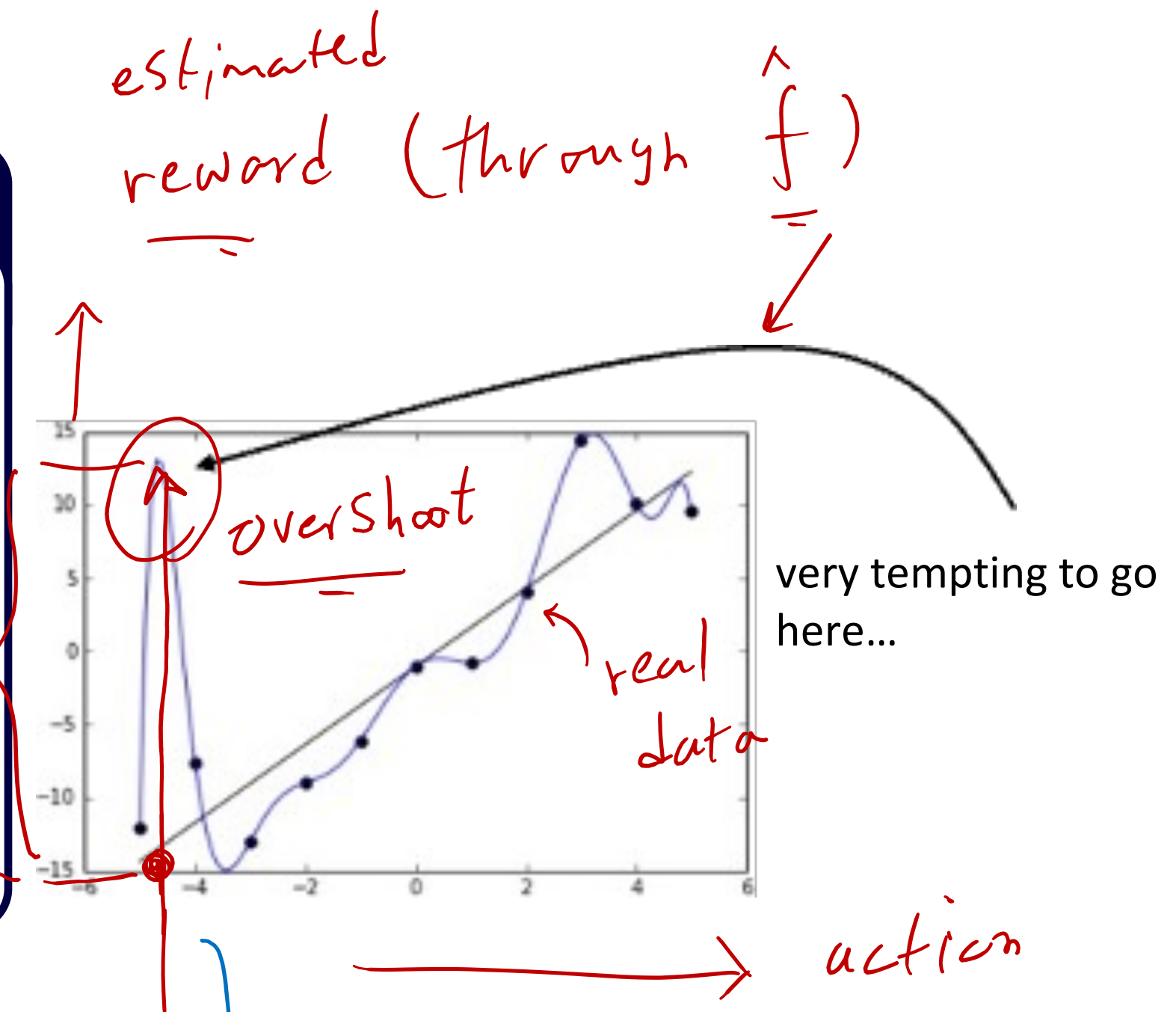
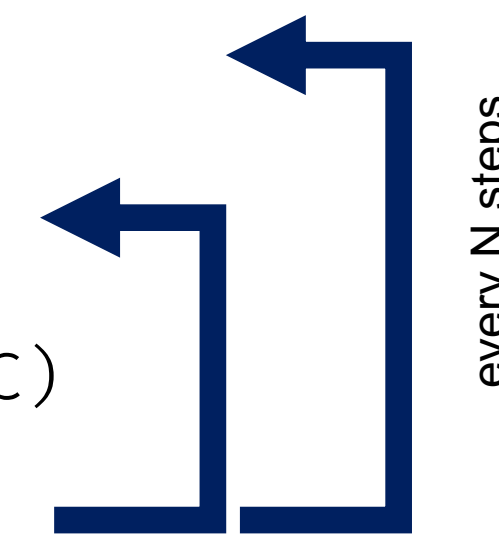
need to not overfit here...

...but still have high capacity over here

Why the Performance gap? (cont.)

Model-Based reinforcement learning v1.5

1. run base policy $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$ (e.g., random policy) to collect $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$
2. learn dynamics model $f(\mathbf{s}, \mathbf{a})$ to minimize $\sum_i \|f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$
3. plan through $f(\mathbf{s}, \mathbf{a})$ to choose actions
4. execute the first planned action, observe resulting state $\mathbf{s}'$ (MPC)
5. append $(\mathbf{s}, \mathbf{a}, \mathbf{s}')$ to dataset $\mathcal{D}$



Q: Based on this insight, how would you modify Alg. v. 1.5?

gap

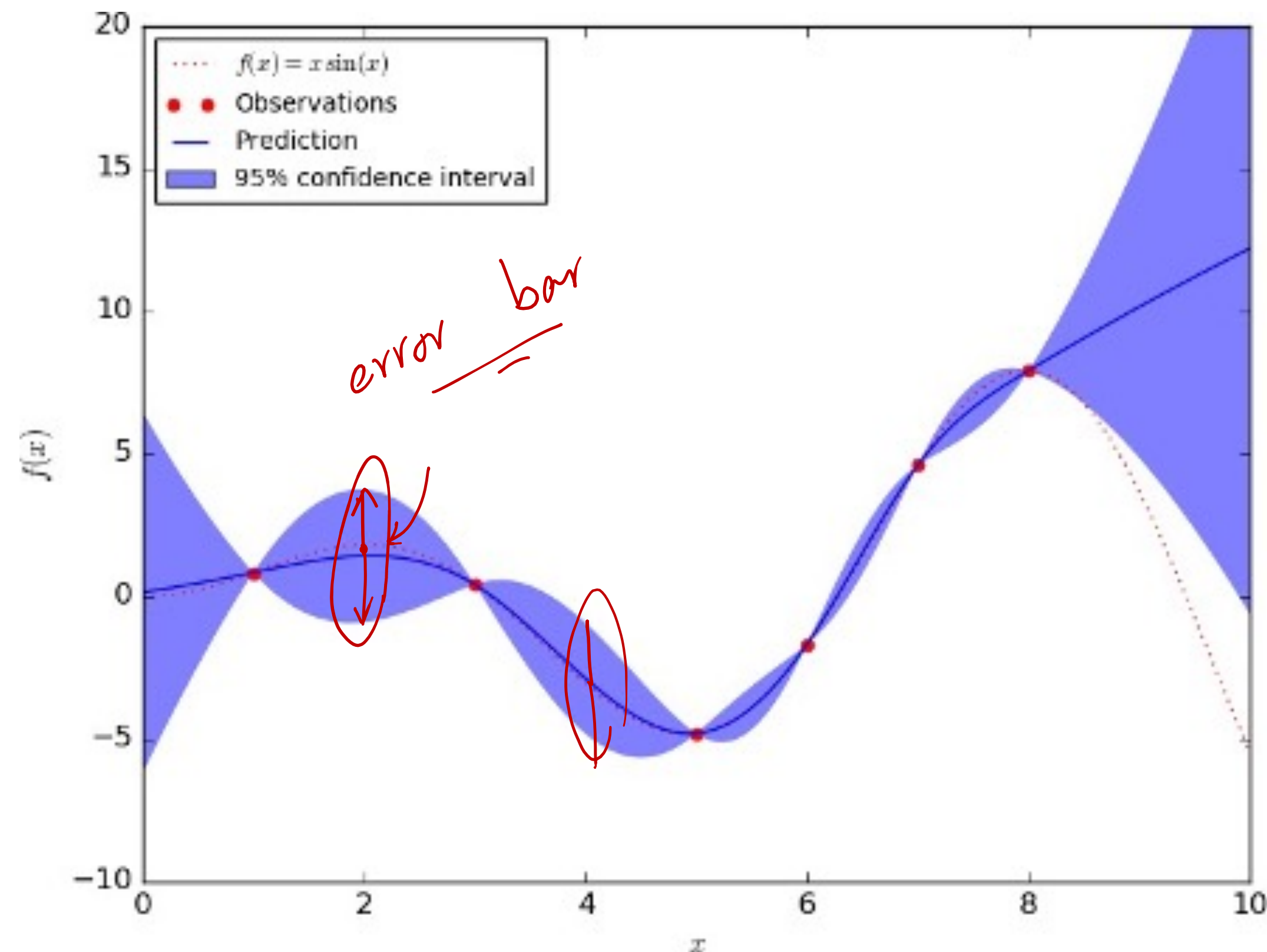
this seems to be an arbitrary behavior

Over shooting gets milder

Average the estimated reward based on k samples

fit k models

How can uncertainty estimation help?



$$P_{\pi_f}(s_t) \neq P_{\pi_0}(s_t)$$



expected reward under high-variance prediction is very low, even though mean is the same!

Intuition behind uncertainty-aware RL

Model-Based reinforcement learning v1.5

1. run base policy $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$ (e.g., random policy) to collect $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$
2. learn dynamics model $f(\mathbf{s}, \mathbf{a})$ to minimize $\sum_i \|f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$
3. plan through $f(\mathbf{s}, \mathbf{a})$ to choose actions
4. execute the first planned action, observe resulting state $\mathbf{s}'$ (MPC)
5. append $(\mathbf{s}, \mathbf{a}, \mathbf{s}')$ to dataset $\mathcal{D}$

only take actions for which we think we'll get high reward in expectation (w.r.t. uncertain dynamics)

This avoids "exploiting" the model

The model will then adapt and get better

Goal: Measure Uncertainty in f .

There are a few caveats...



Need to explore to get better

Expected value is not the same as pessimistic value

Expected value is not the same as optimistic value

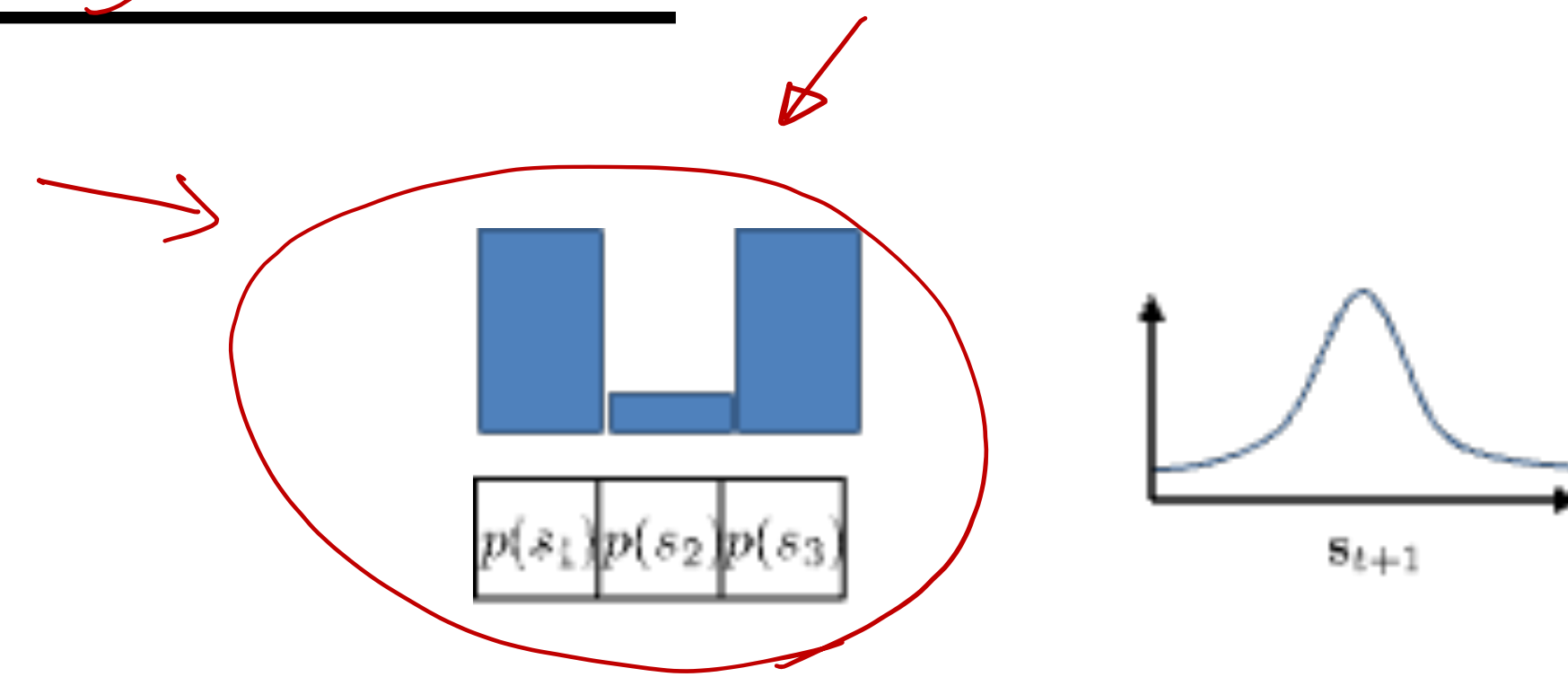
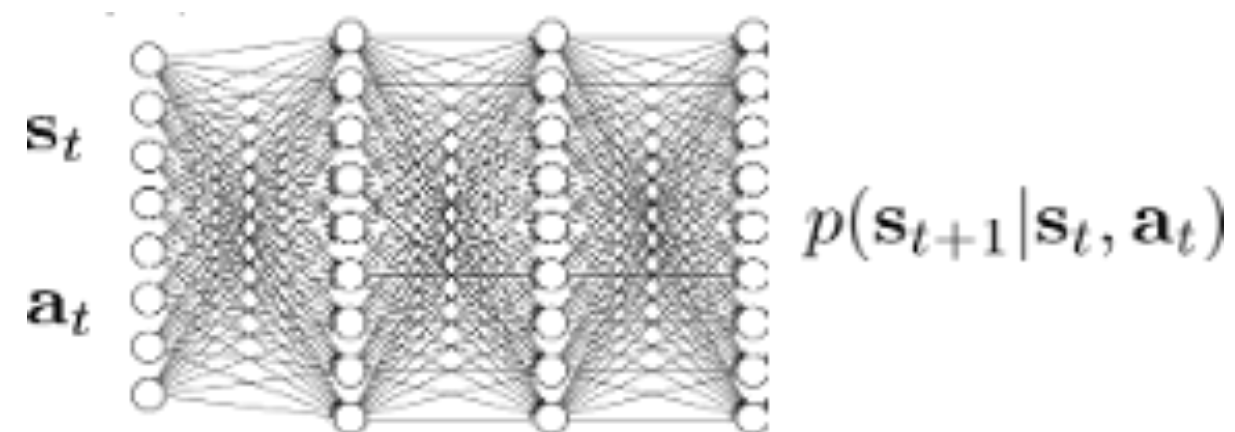
...but expected value is often a good start

Uncertainty-Aware Neural Net Models

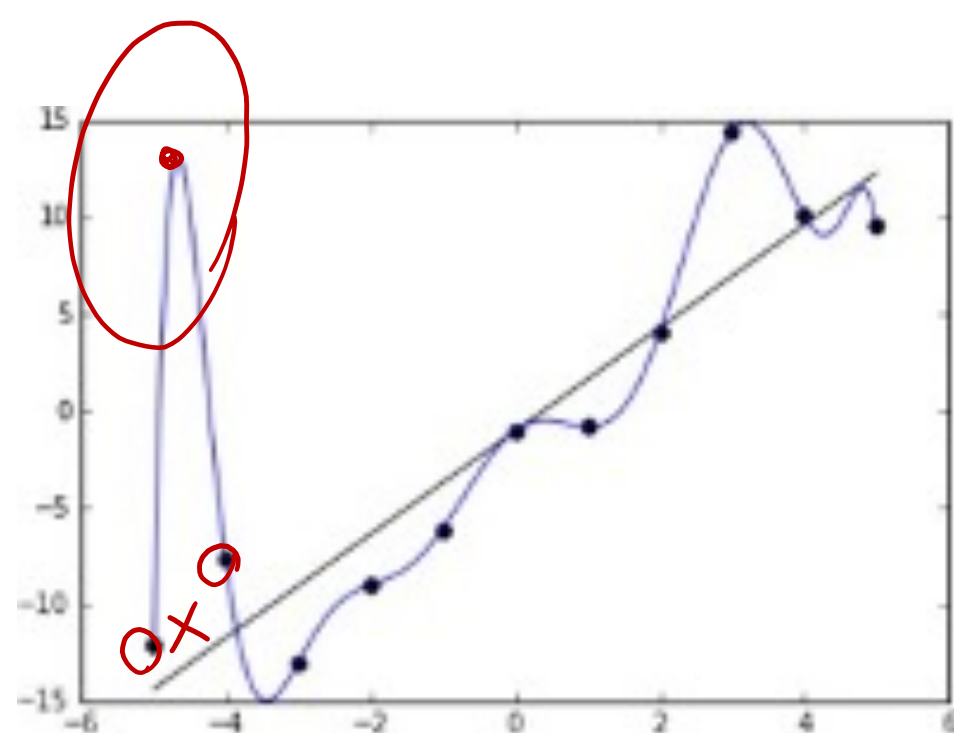
How can we have uncertainty-aware Models?

$R = \sum_{t=1}^T r(s_t, a_t)$
 $s_t = f(s_{t-1}, a_{t-1})$

Idea 1: use output entropy



why is this not enough?

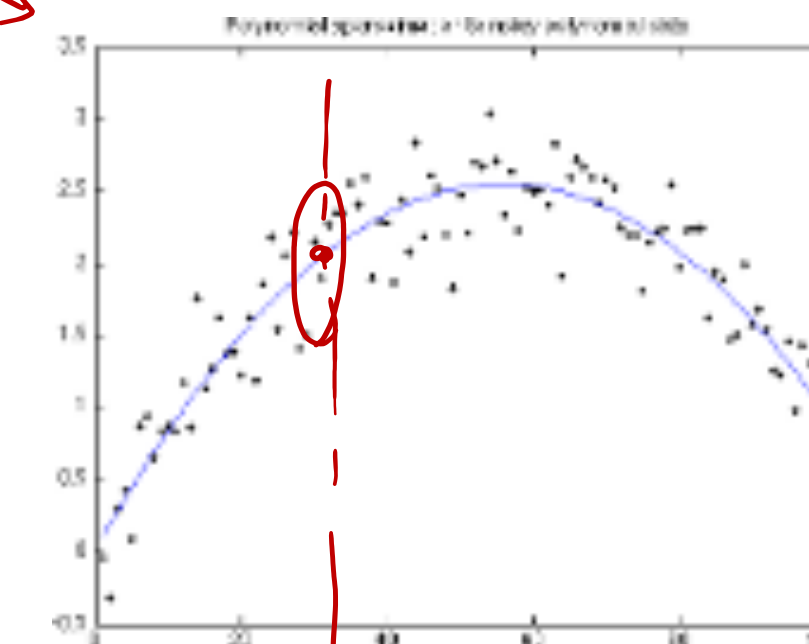


Two types of uncertainty:

① aleatoric or statistical uncertainty

② epistemic or model uncertainty

lack of data



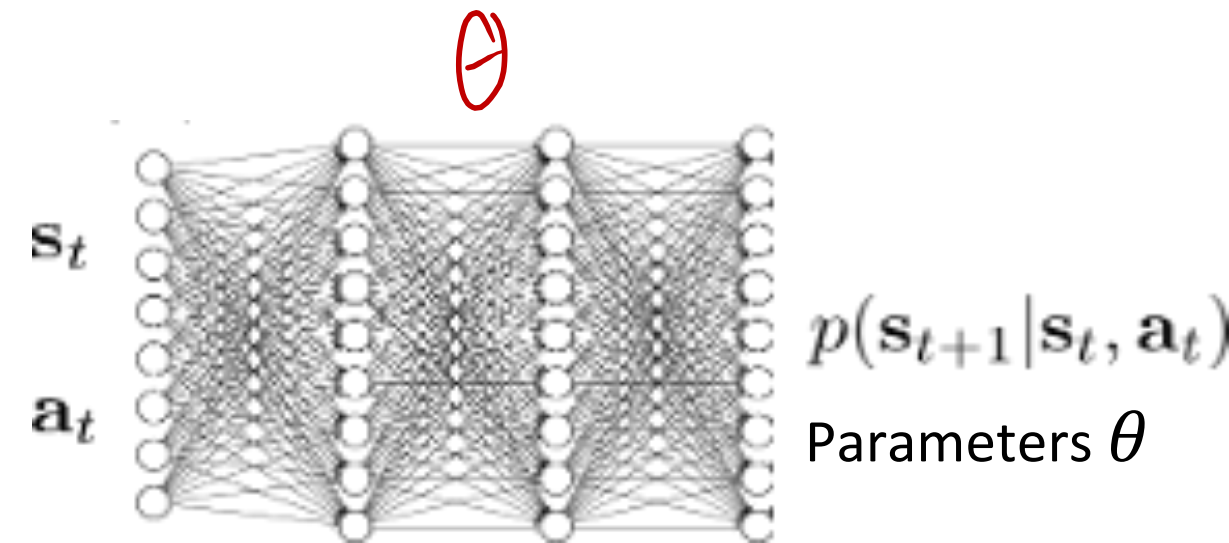
what is the variance here?

“the model is certain about the data, but we are not certain about the model”

How can we have uncertainty-aware Models? (cont.)

$p(\theta) \cdot D \rightarrow P(\theta|D) = \frac{P(D|\theta) \cdot P(\theta)}{P(D)}$

Idea 2: estimate model uncertainty



θ^*
 $P(\theta|D)$
 posterior

Usually, we estimate

$$\operatorname{argmax}_{\theta} \log p(\theta|D) = \operatorname{argmax}_{\theta} \log p(D|\theta)$$

Can we instead estimate $P(\theta|D)$?

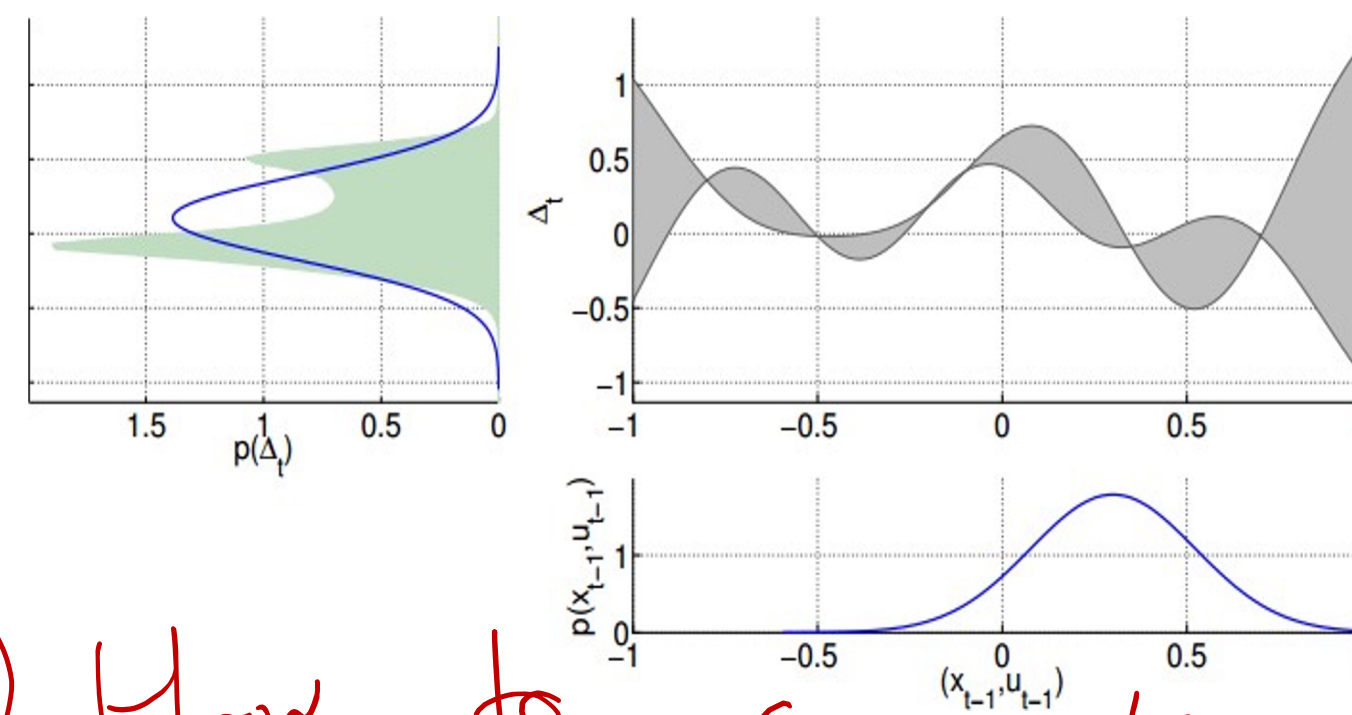
"the model is certain about the data, but we are not certain about the model"

the entropy of this tells us the model uncertainty!

Predict according to:

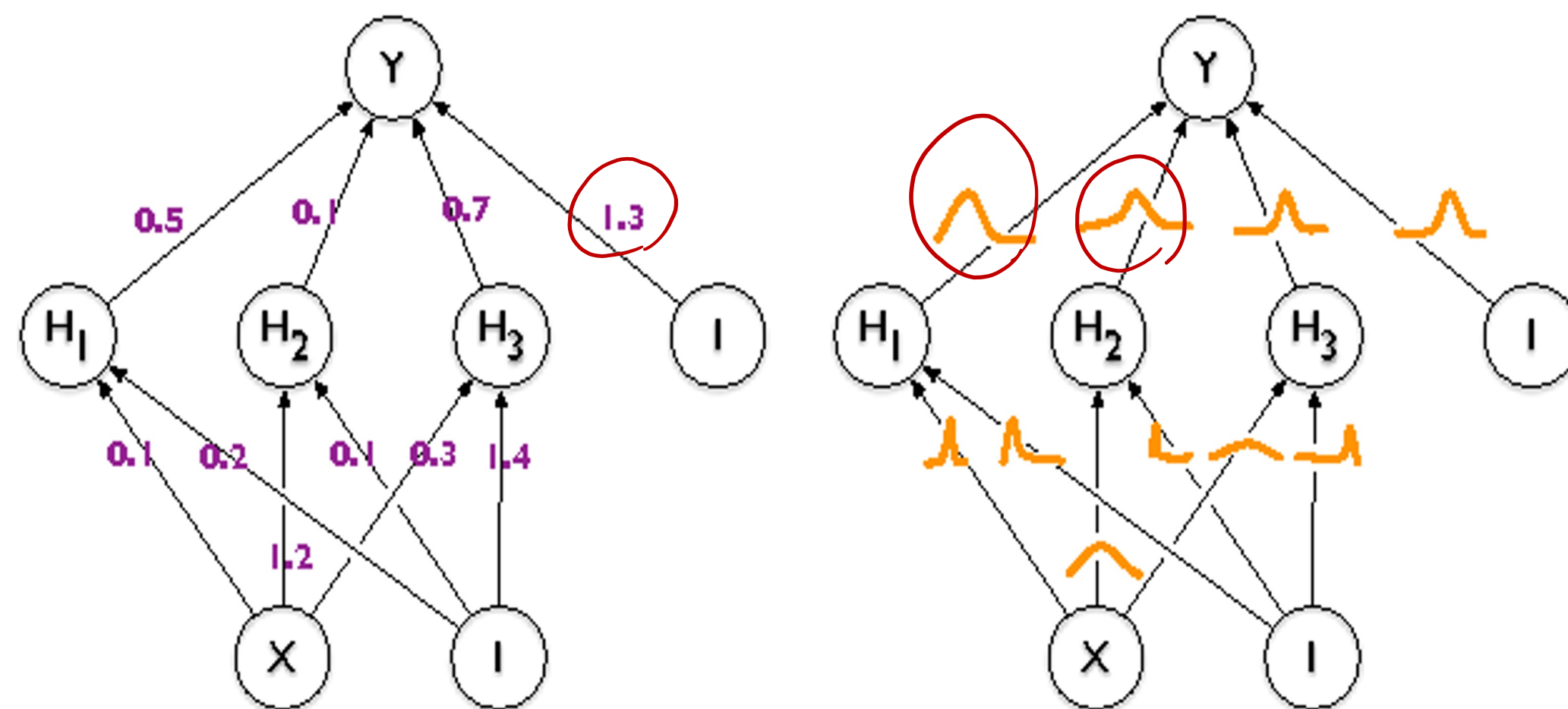
$$\int \underbrace{p(s_{t+1} | s_t, a_t, \theta)}_f \underbrace{p(\theta|D)}_d d\theta$$

Avg.



(2) How to compute the integral?

Quick overview of Bayesian Neural Networks!



Common approximation:

$$p(\theta|\mathcal{D}) = \prod_i p(\theta_i|\mathcal{D})$$

$$p(\theta_i|\mathcal{D}) = \mathcal{N}(\mu_i, \sigma_i)$$

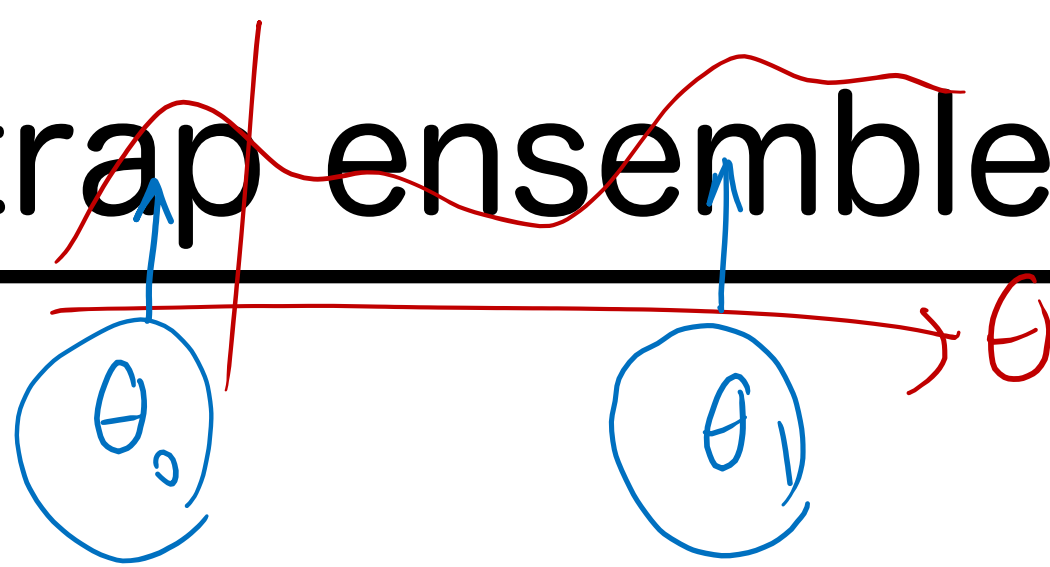
How to train?

For more, see:

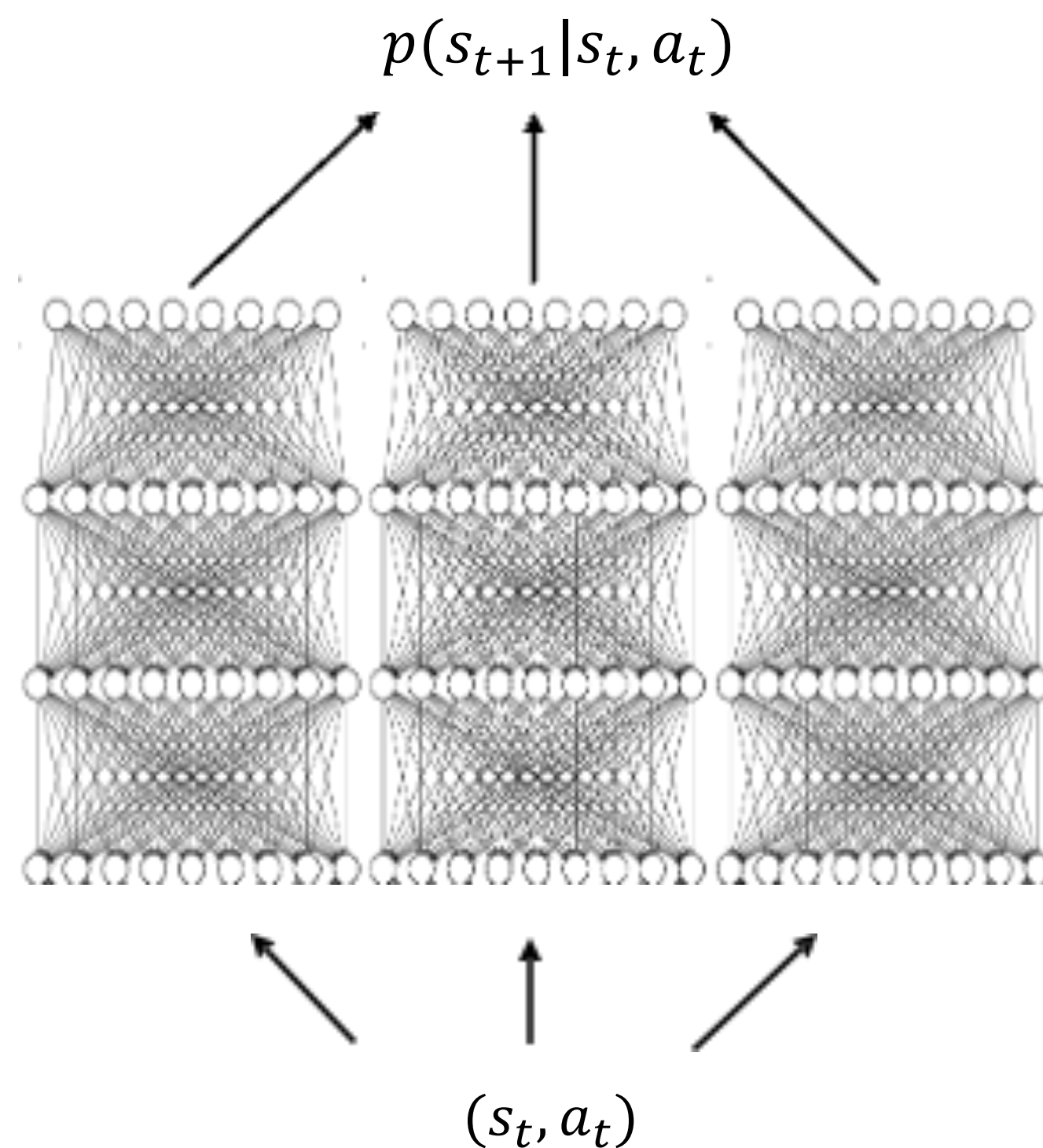
Blundell et al., Weight Uncertainty in Neural Networks Gal et al., Concrete Dropout

Bootstrap ensembles

$$P(\theta | \mathcal{D}) = \frac{1}{N} \sum_{i=1}^N \delta(\theta - \theta_i)$$



- ◆ Train multiple models and see if they agree!



formally: $p(\theta | \mathcal{D}) \approx \frac{1}{N} \sum_i \delta(\theta_i)$

① $\int p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t, \theta) \underbrace{p(\theta | \mathcal{D})}_{\delta(\theta - \theta_i)} d\theta \approx \frac{1}{N} \sum_i \underbrace{p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t, \theta_i)}_{\text{Avg.}}$

- ◆ How to train?

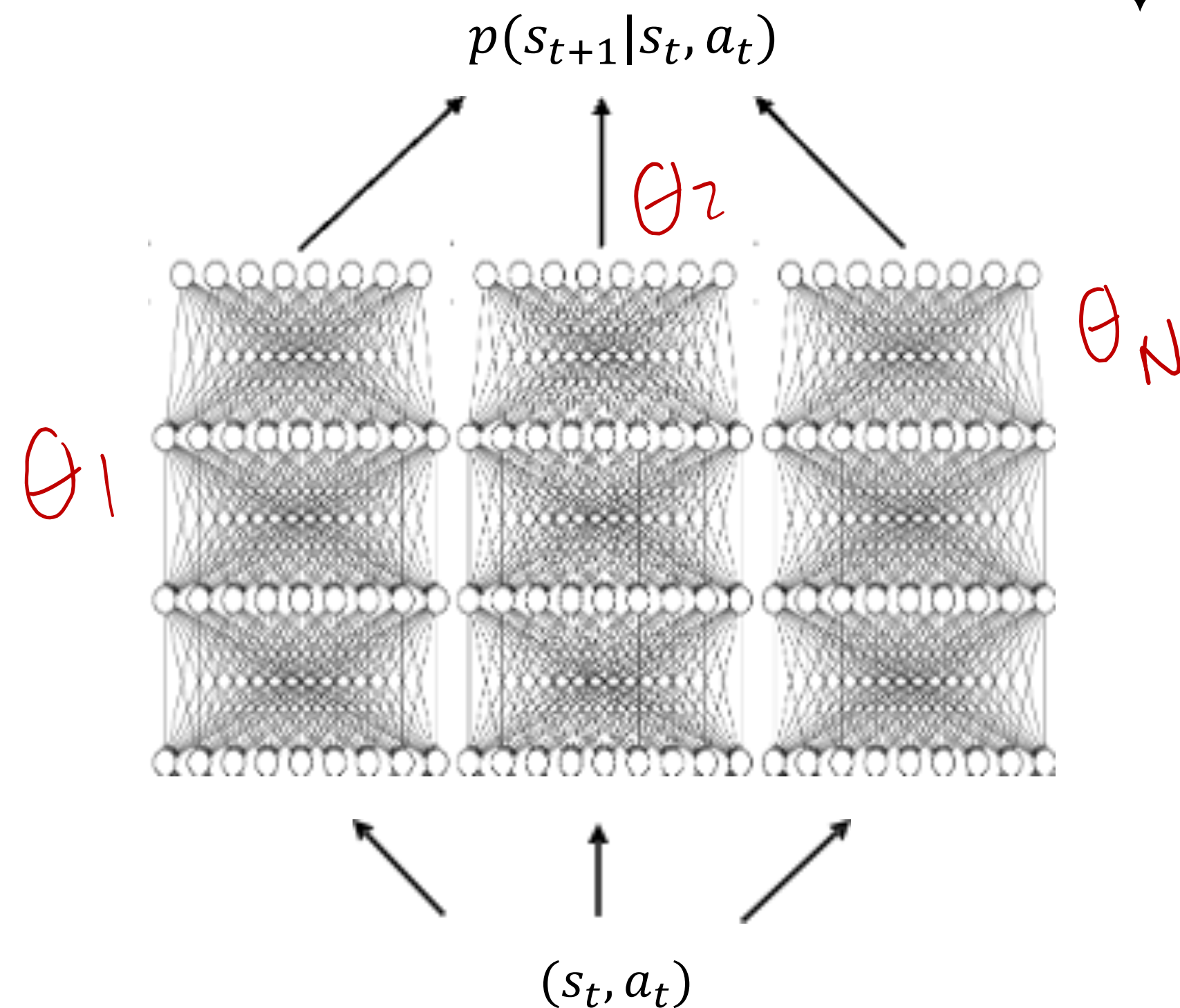
- ✓ Main idea: need to generate “independent” datasets to get “independent” models

θ_i is trained on D_i , sampled with replacement from $\mathcal{D}$

$$\hat{f}_1 \dots \hat{f}_N$$

Bootstrap ensembles in Deep Learning

♦ This basically works



Very crude approximation, because the number of models is usually small (< 10)

Resampling with replacement is usually unnecessary, because SGD and random initialization usually makes the models sufficiently independent

Planning with Uncertainty, Examples

How to Plan with Uncertainty?

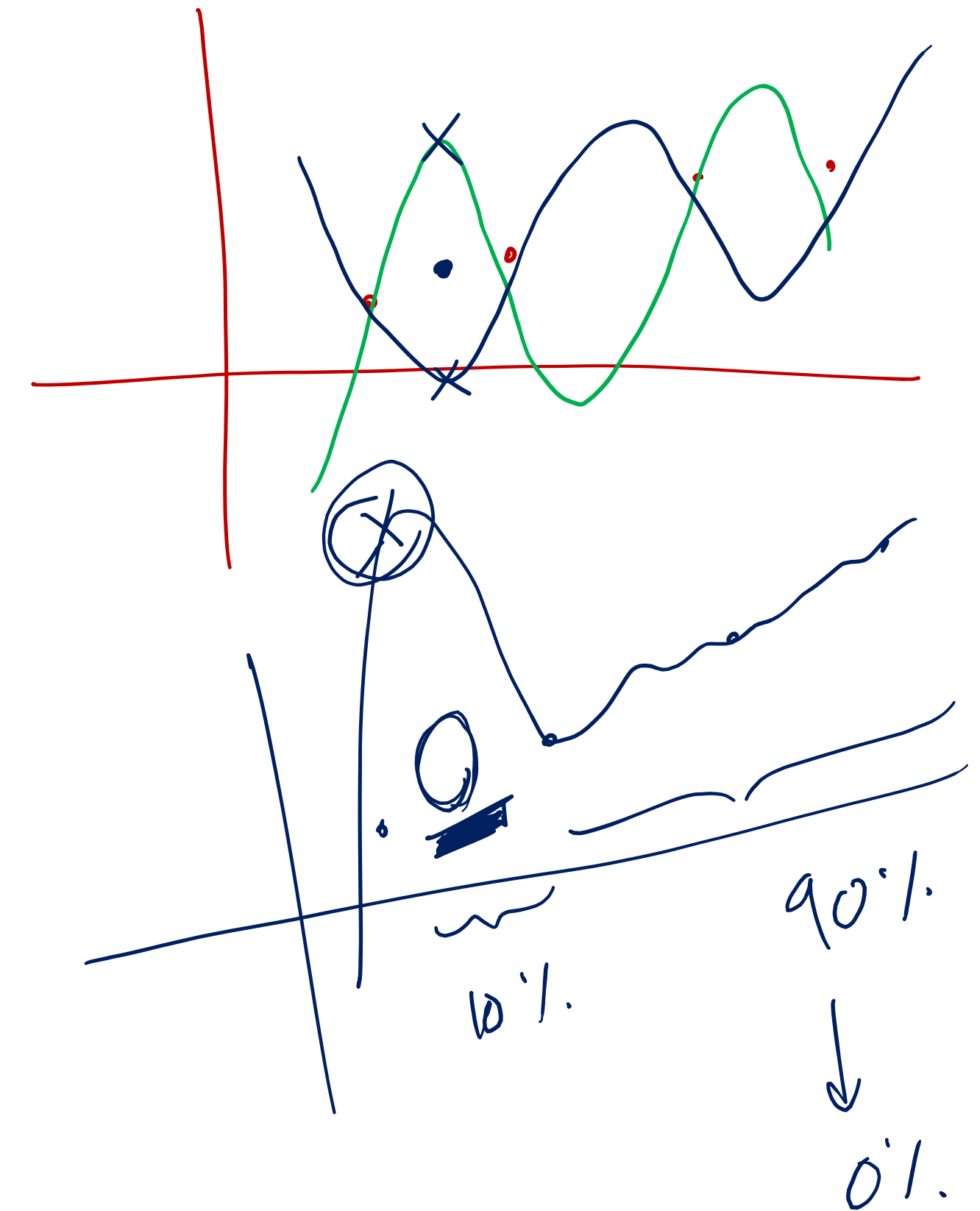
Before: $J(\mathbf{a}_1, \dots, \mathbf{a}_H) = \sum_{t=1}^H r(\mathbf{s}_t, \mathbf{a}_t)$, where $\mathbf{s}_{t+1} = f(\mathbf{s}_t, \mathbf{a}_t)$

Now: $J(\mathbf{a}_1, \dots, \mathbf{a}_H) = \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H r(\mathbf{s}_{t,i}, \mathbf{a}_t)$, where $\mathbf{s}_{t+1,i} = f_i(\mathbf{s}_{t,i}, \mathbf{a}_t)$

In general, for candidate action sequence $\mathbf{a}_1, \dots, \mathbf{a}_H$:

1. Step 1: sample $\theta \sim p(\theta|\mathcal{D})$
2. Step 2: at each time step t , sample $\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t, \theta)$
3. Step 3: calculate $R = \sum_t r(\mathbf{s}_t, \mathbf{a}_t)$
4. Step 4: repeat steps 1 to 3 and accumulate the average reward

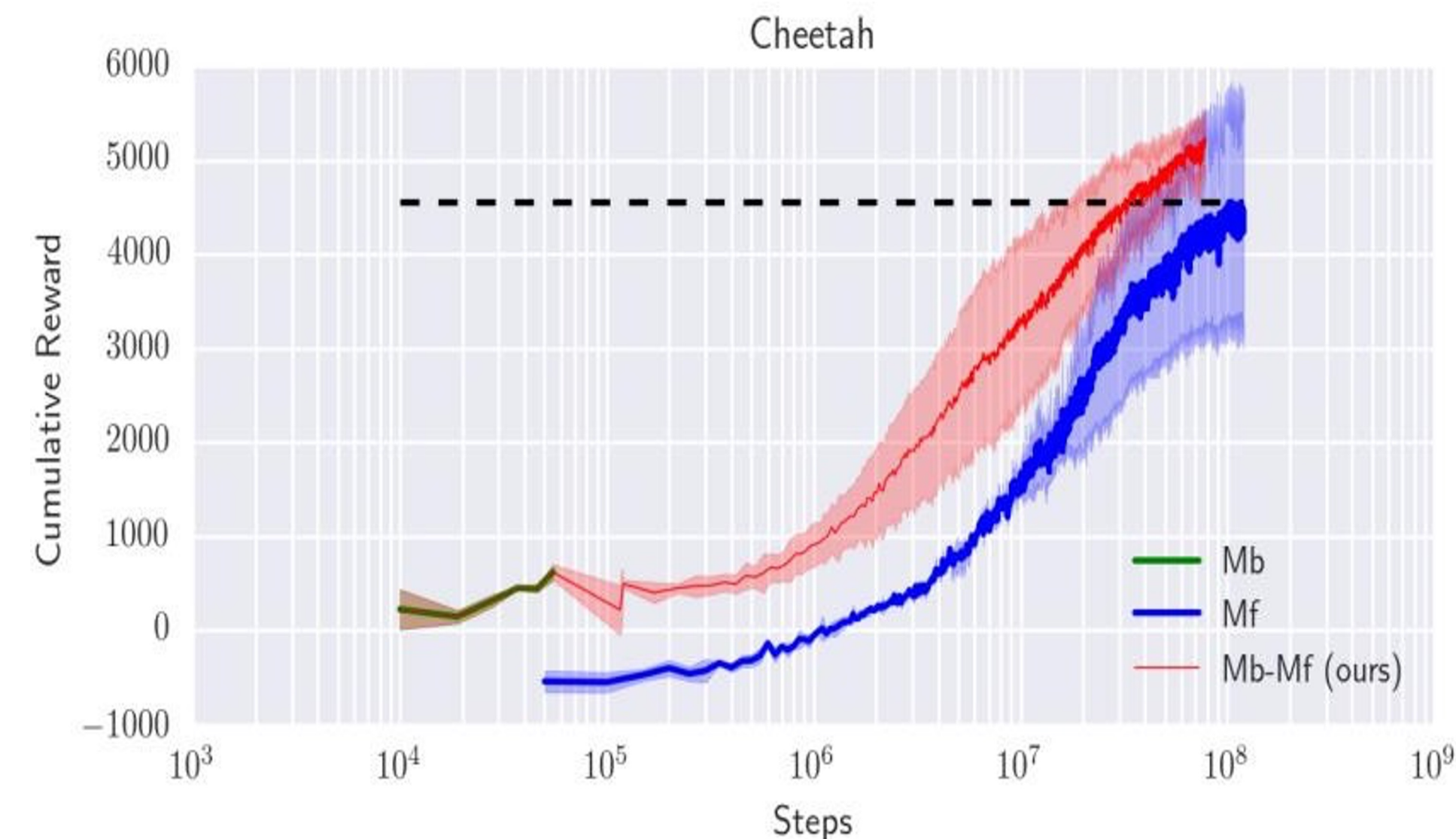
Other options: moment matching, more complex posterior estimation with BNNs, etc.



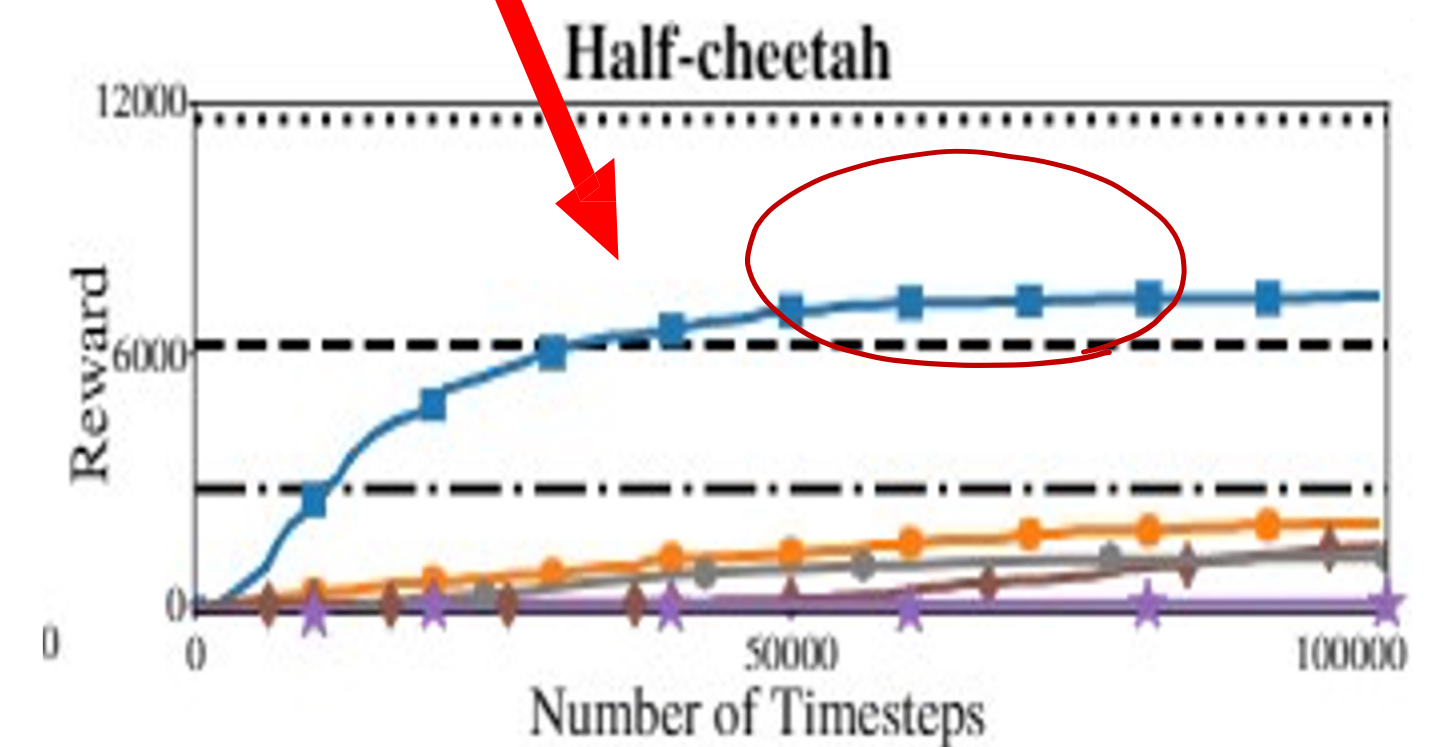
Example: Model-Based RL with ensemble!

Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models

exceeds performance of model-free after 40k steps (about 10 minutes of real time)

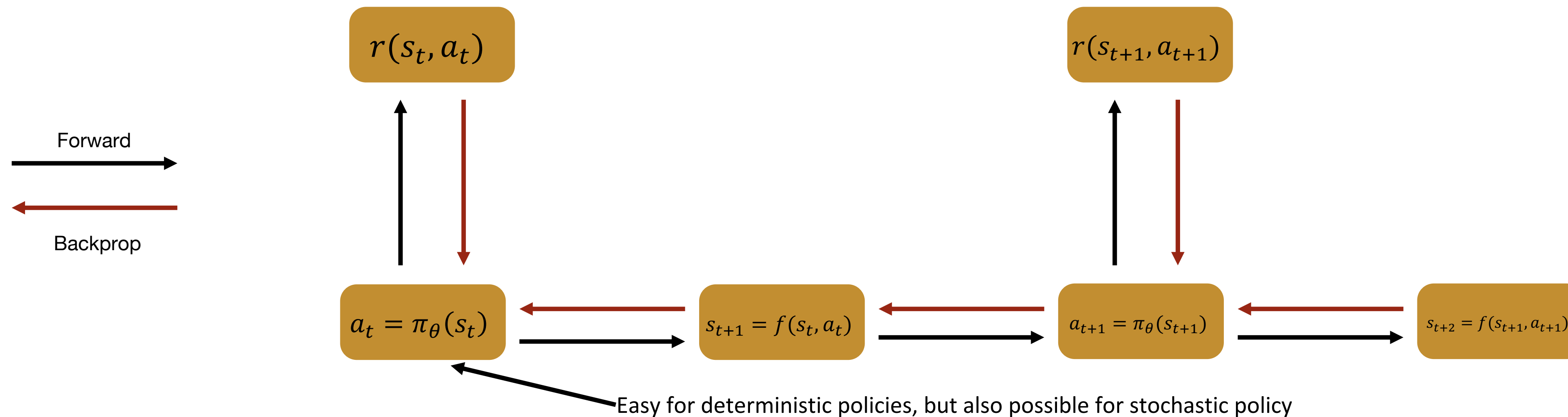


before



after

Backpropagate directly into the policy?

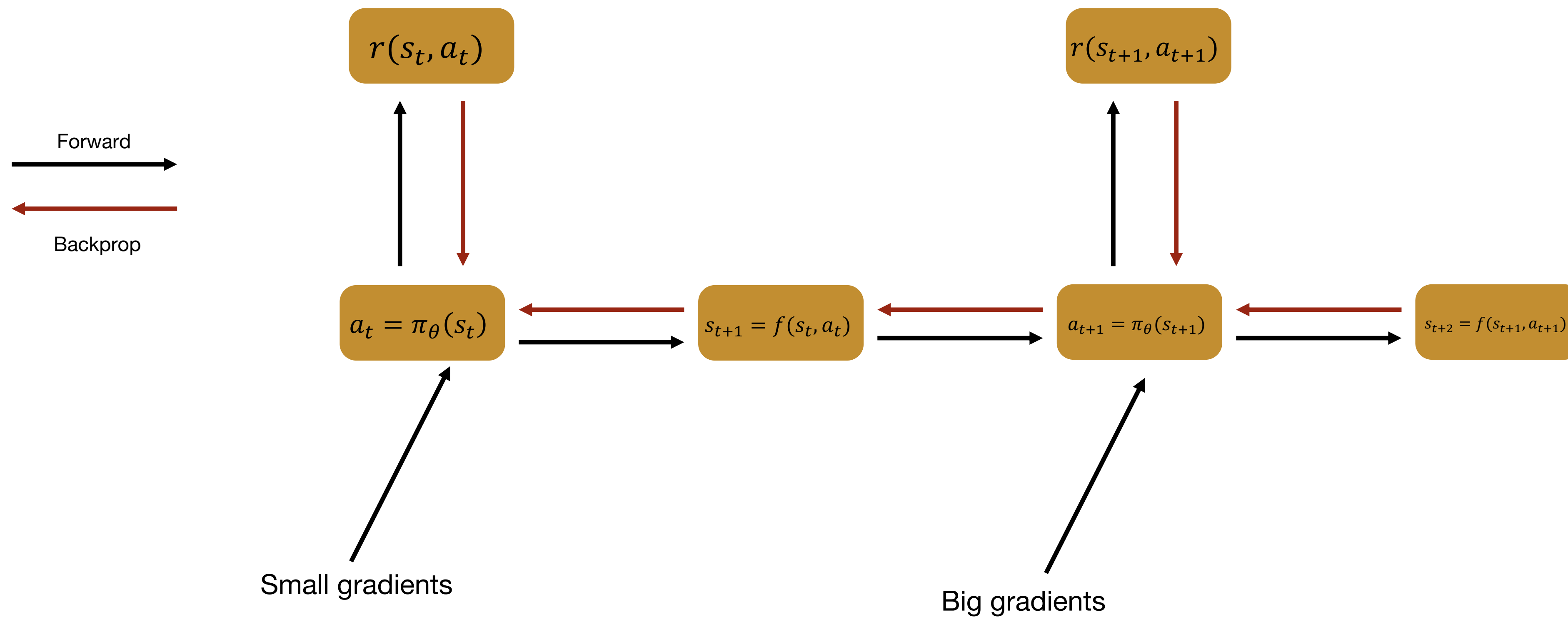


Model-Based reinforcement learning v2

1. run base policy $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$ (e.g., random policy) to collect $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$
2. learn dynamics model $f(\mathbf{s}, \mathbf{a})$ to minimize $\sum_i \|f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$
3. backpropagate through $f(\mathbf{s}, \mathbf{a})$ into the policy to optimize $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$
4. run $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$, appending the visited tuples $(\mathbf{s}, \mathbf{a}, \mathbf{s}')$ to $\mathcal{D}$



What's the problem with backprop into Policy?



What's the problem with backprop into Policy? (cont.)

- ♦ Similar problems to training long RNNs with BPTT
 - ☑ **Vanishing** and **exploding** gradients
 - ☑ Unlike LSTM, we can't just "choose" a simple dynamics, **dynamics are chosen by nature**

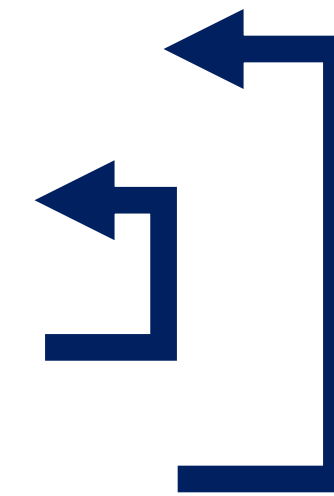
What's the problem with backprop into Policy? (cont.)

- ✦ Use derivative-free (“model-free”) RL algorithms, with **the model used to generate synthetic samples**
- ✦ Seems weirdly backwards
 - ☑ Actually works very well
 - ☑ Essentially “model-based acceleration” for model-free RL

Model-Based RL with policy gradient

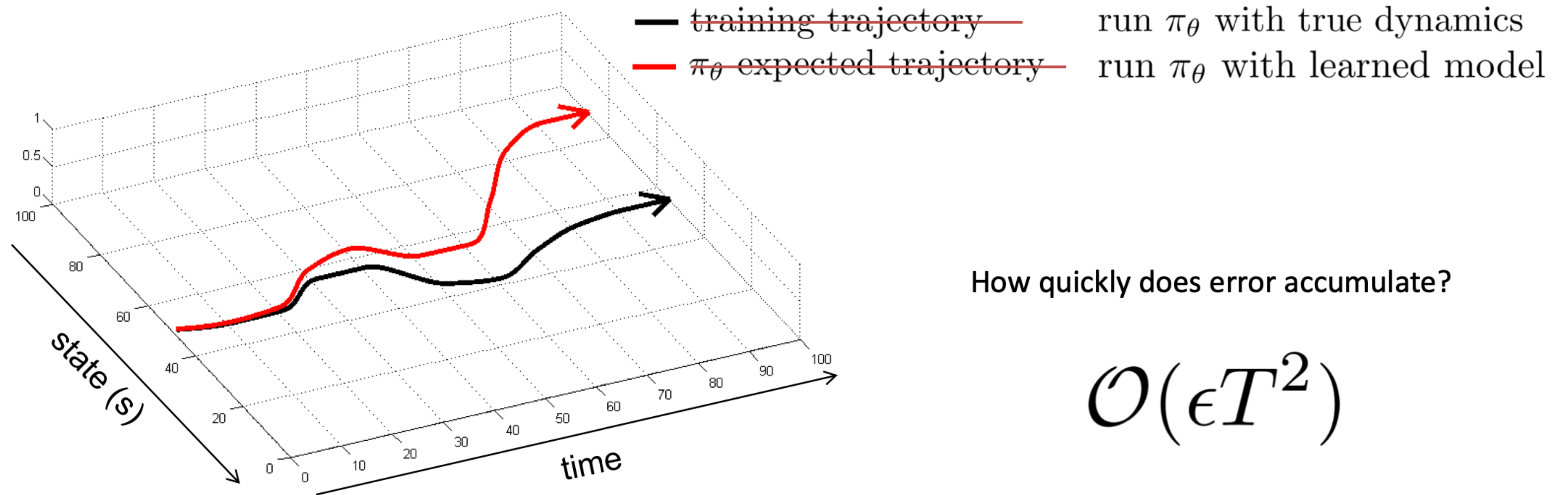
Model-Based reinforcement learning v2.5

1. run base policy $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$ (e.g., random policy) to collect $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$
2. learn dynamics model $f(\mathbf{s}, \mathbf{a})$ to minimize $\sum_i \|f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$
3. use $f(\mathbf{s}, \mathbf{a})$ to generate trajectories $\{\tau_i\}$ with policy $\pi_\theta(\mathbf{a}|\mathbf{s})$
4. use $\{\tau_i\}$ to improve $\pi_\theta(\mathbf{a}|\mathbf{s})$ via policy gradient
5. run $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$, appending the visited tuples $(\mathbf{s}, \mathbf{a}, \mathbf{s}')$ to $\mathcal{D}$

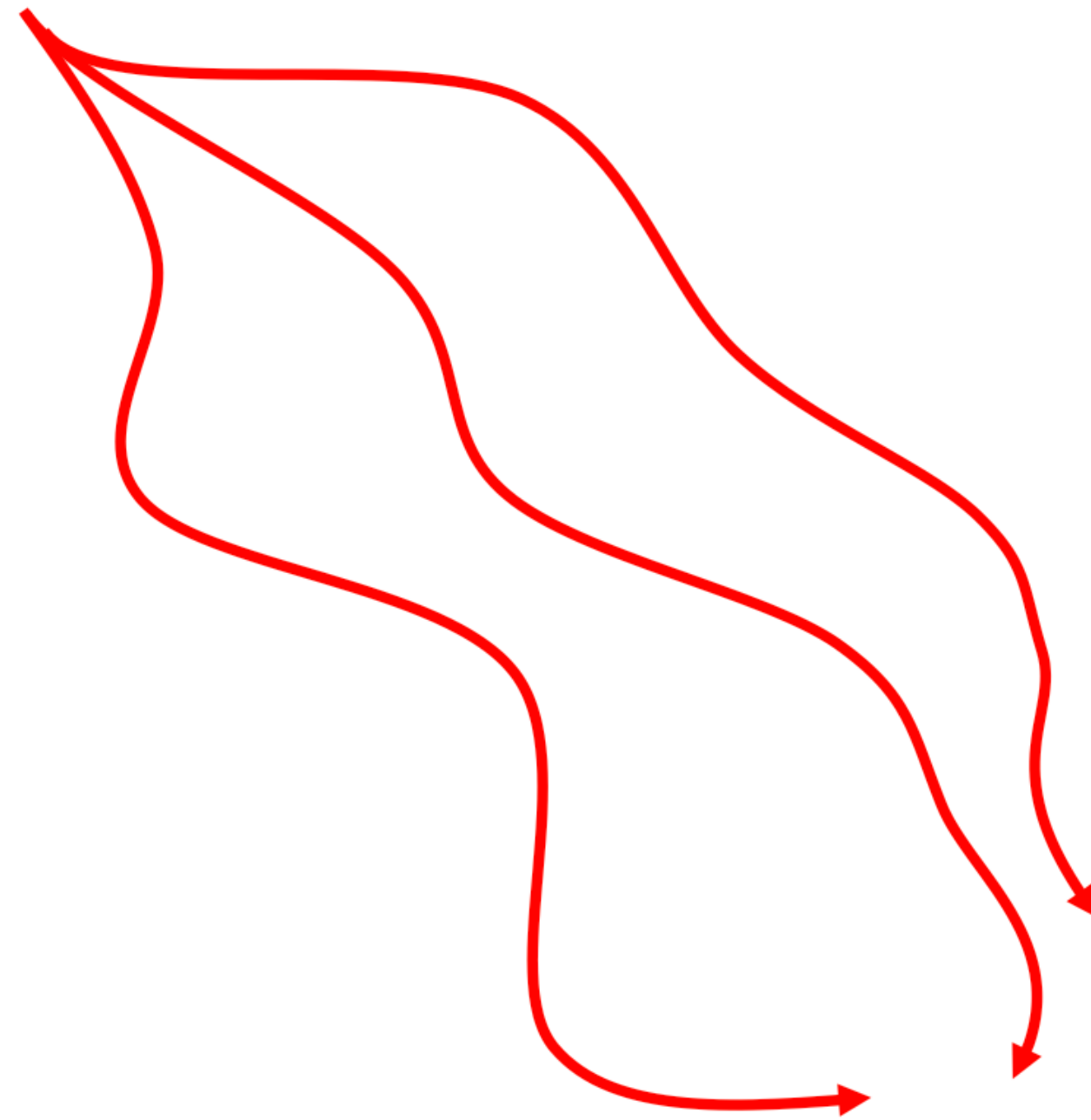


What's a potential problem with this approach?

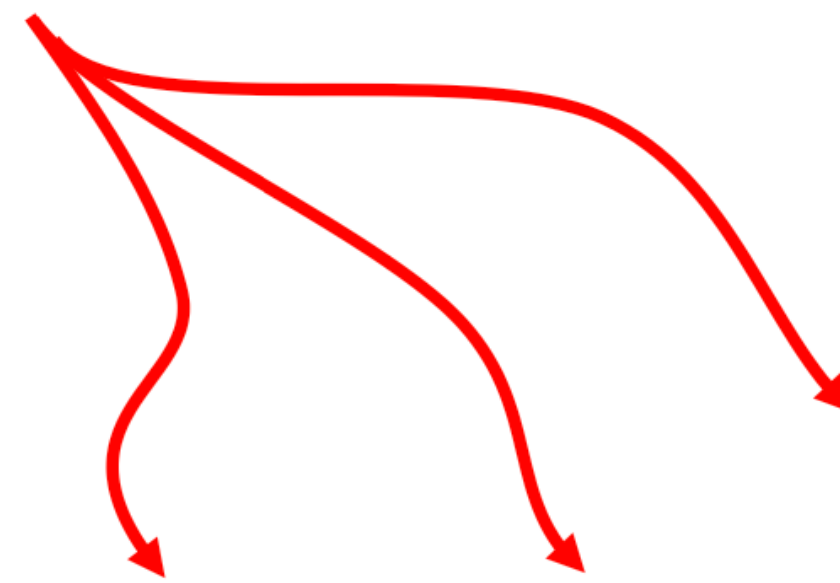
The curse of long Model-Based rollouts



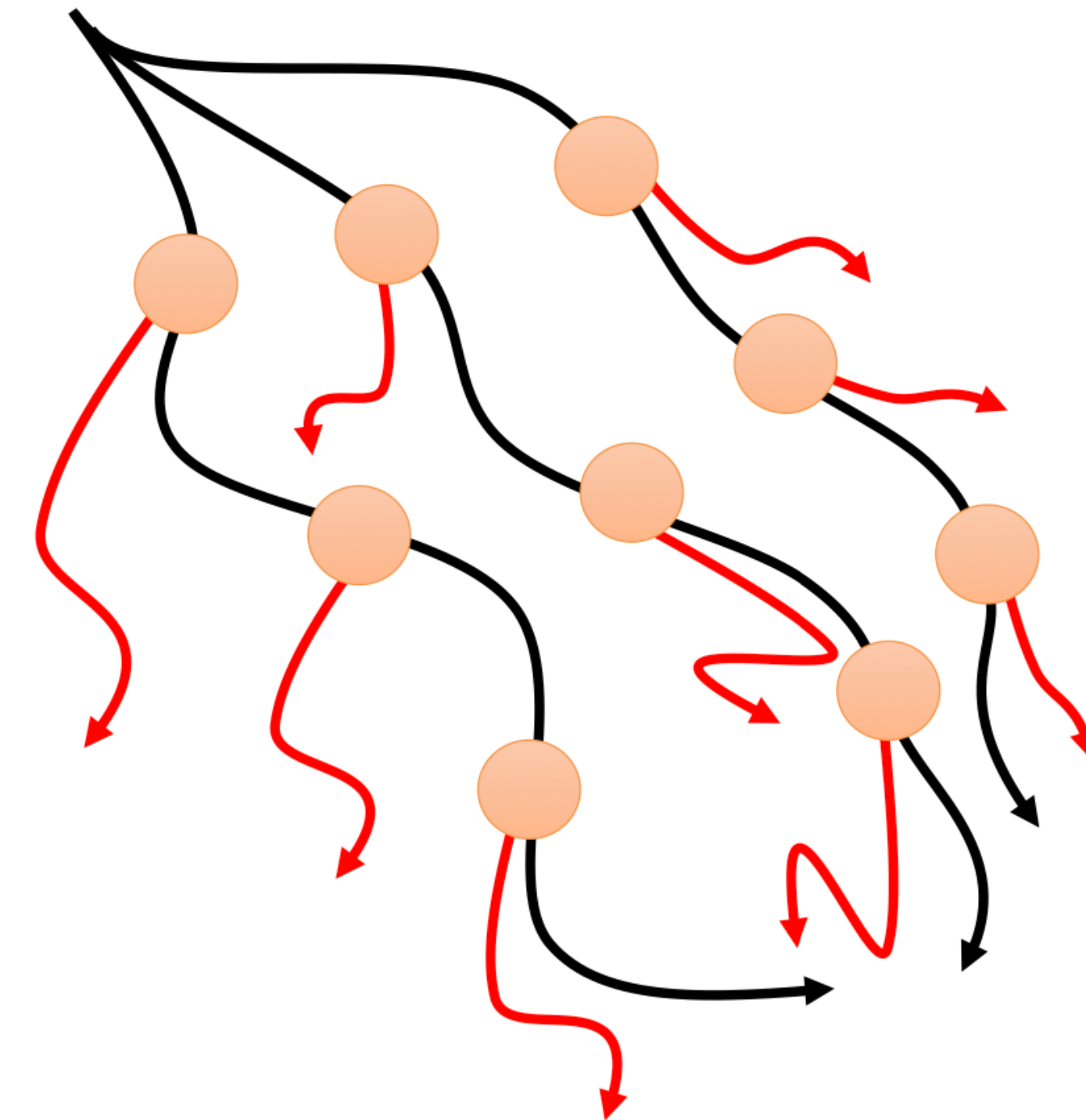
How to get away with accumulated errors?



- huge accumulating error



+ much lower error
- never see later time steps

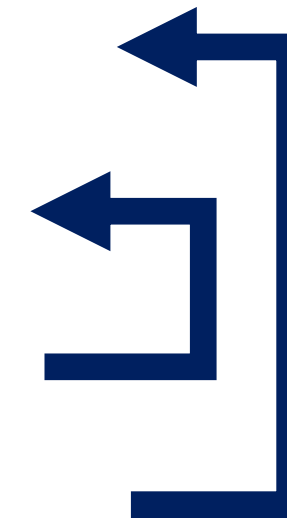


+ much lower error
+ see all time steps
- wrong state distribution

Model-Based RL with short rollouts

Model-Based reinforcement learning v3

1. run base policy $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$ (e.g., random policy) to collect $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$
2. learn dynamics model $f(\mathbf{s}, \mathbf{a})$ to minimize $\sum_i \|f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$
3. pick states $\mathbf{s}_i$ from $\mathcal{D}$, use $f(\mathbf{s}, \mathbf{a})$ to make short rollouts from them
4. use both real and model data to improve $\pi_\theta(\mathbf{a}|\mathbf{s})$ with off-policy RL
5. run $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$, appending the visited tuples $(\mathbf{s}, \mathbf{a}, \mathbf{s}')$ to $\mathcal{D}$



Dyna

Dyna-Q Algorithm

1. given state s , pick action a using exploration policy
2. observe s' and r , to get transition (s, a, s', r)
3. update model $\hat{p}(s'|s, a)$ and $\hat{r}(s, a)$ using (s, a, s')
4. Q-update: $Q(s, a) \leftarrow Q(s, a) + \alpha E_{s', r} [r + \max_{a'} Q(s', a') - Q(s, a)]$
5. repeat K times:
 6. sample $(s, a) \sim \mathcal{B}$ from buffer of past states and actions
 7. Q-update: $Q(s, a) \leftarrow Q(s, a) + \alpha E_{s', r} [r + \max_{a'} Q(s', a') - Q(s, a)]$

Dyna (cont.)

Dyna-Q Algorithm - Direct RL & Planning

Loop forever:

- (a) $S \leftarrow \text{current (non-terminal) state}$
- (b) $A \leftarrow \epsilon\text{-greedy}(S, Q)$
- (c) *Take action A; observe resultant reward, R, and state, S'*
- (d) $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$
- (e) $\text{Model}(S, A) \leftarrow R, S'$ (*assuming deterministic environment*)
- (f) *Loop repeat k times:*
 - $S \leftarrow \text{random previously observed state}$
 - $A \leftarrow \text{random action previously taken in } S$
 - $R, S' \leftarrow \text{Model}(S, A)$
 - $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$

General “Dyna-Style” Model-Based RL recipe

Dyna Algorithm

1. collect some data, consisting of transitions (s, a, s', r)
2. learn model $\hat{p}(s'|s, a)$ (and optionally, $\hat{r}(s, a)$)
3. repeat K times:
 4. sample $s \sim \mathcal{B}$ from buffer
 5. choose action a (from $\mathcal{B}$, from π , or random)
 6. simulate $s' \sim \hat{p}(s'|s, a)$ (and $r = \hat{r}(s, a)$)
 7. train on (s, a, s', r) with model-free RL
 8. (optional) take N more model-based steps

- + only requires short (as few as one step) rollouts from model
- + still sees diverse states